

一步即足够：分散平均流策略优化

Guowei Zou^{1,2} Haitao Wang^{1,2} Hejun Wu^{1,2} Yukun Qian^{1,2} Yuhang Wang^{1,2} Weibing Li^{1,2}

摘要

实时机器人控制要求快速生成动作。然而，现有基于扩散和流匹配的生成式策略需要多步采样，从根本上限制了其在时间关键场景中的部署。本文提出分散平均流策略优化（Dispersive MeanFlow Policy Optimization, **DMPO**），一个统一框架，通过三个关键组件实现真正的一步生成：平均流（MeanFlow）用于无需知识蒸馏的数学推导单步推理，分散正则化防止表示坍塌，以及强化学习（RL）微调以超越专家演示。在 RoboMimic 操作和 OpenAI Gym 运动基准上的实验表明，与多步基线相比，性能具有竞争力或更优。凭借轻量级模型架构和三个关键算法组件的协同作用，**DMPO** 满足实时控制要求（ $>120\text{Hz}$ ），推理速度提升 $5-20\times$ 倍，在高性能 GPU 上达到数百赫兹。在 Franka-Emika-Panda 机器人上的物理部署验证了其现实世界适用性。代码可在 <https://guowei-zou.github.io/dmpo-page/> 获取。

1. 引言

实时控制对于灵巧机器人操作至关重要，需要快速生成动作以确保平滑和响应性行为。基于扩散和流匹配的生成模型在建模复杂、多模态动作分布方面取得了显著成功（Chi 等人，2023；Ze 等人，2024；Zhang 等人，2025a），然而其多步采样过程导致每个动作的显著延迟，从根本上限制了在时间关键场景中的部署。1 中山大学计算机科学与工程学院，中国广州 2 广东省大数据分析处理重点实验室，中国广州。邮箱：{zougw, qianyk5, wangyh253}@mail.sysu.edu.cn；{wanght39, wuhejun, liwb53}@mail.sysu.edu.cn。

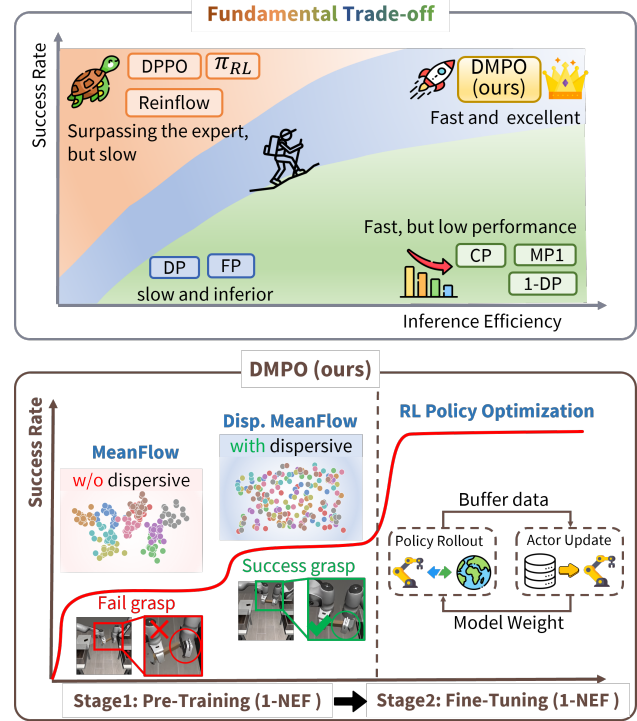


图 1. 从效率-性能权衡到实用的实时控制。顶部：现有方法位于权衡曲线上：多步方法（DPPO、ReinFlow、 π_{RL} ）性能强但推理慢，而一步方法（CP、MP1、1-DP）速度快但不稳定。DMPO 通过占据右上区域打破了这一权衡。底部：DMPO 的两阶段方法：分散平均流在预训练期间防止表示坍塌，PPO 微调实现超越专家的适应。

场景。这促使人们追求能够匹配多步质量、同时支持实时控制的一步生成策略。然而，现有方法面临三个相互关联的挑战：（1）推理效率，扩散和基于流的策略中的多步采样会带来显著延迟，而基于蒸馏的一步方法如 CP（Prasad 等人，2024）和 1-DP（Wang 等人，2025）需要复杂的训练流程（Lu 等人，2024；Yan 等人，2025；Clemente 等人，2025）；（2）表示坍塌，一步生成方法可能将不同观测映射到不可区分的表示，从而降低动作质量，且没有机会进行迭代

修正 (Wang & He, 2025; Zou 等人, 2026); (3) 性能上限, 因为纯模仿学习无法超越专家演示, 而强化学习 (RL) 微调 (Ren 等人, 2025; Zhang 等人, 2025b; Chen 等人, 2025b) 在缓慢的多步推理下不切实际, 且在表示坍塌时不稳定。这些挑战形成一个耦合循环: 快速推理需要一步生成, 稳定的一步生成需要防止坍塌, 突破性能上限需要 RL 微调, 而高效的 RL 微调需要快速推理和高质量表示。任何只解决一个挑战的方法都会被其他挑战所阻碍, 如图 1 所示。

本文提出色散平均流策略优化 (Dispersive MeanFlow Policy Optimization, DMPO), 通过三个相互支持的组件解决这一耦合问题。平均流单步生成 (Geng 等, 2025a) 无需蒸馏即可提供数学上严格的单步推理, 为实时控制和高效微调奠定基础。色散正则化 (Wang 与 He, 2025) 防止表征坍塌, 确保单步生成稳定性, 同时为后续强化学习微调提供高质量初始化。强化学习微调 (Zhang 等, 2025b) 突破模仿学习的性能上限, 使策略能够超越专家数据, 而单步推理效率使大规模微调变得实用。这些组件形成正反馈循环: 色散正则化使单步生成稳定, 单步生成使强化学习微调高效, 高质量预训练为强化学习提供良好起点, 最终实现既快速又有效的单步策略。

本文贡献如下: (1) 框架: 本文提出 DMPO, 这是首个突破生成式机器人策略中效率与性能权衡的框架, 在匹配或超越多步基线的同时实现 $5-20 \times$ 倍加速。(2) 理论: 本文建立首个信息论基础, 证明分散正则化对于稳定的一步生成是必要的 (附录 D), 并推导出一步策略强化学习微调的首个数学公式 (附录 C)。(3) 验证: 本文在 RoboMimic (Mandlekar 等人, 2021) 和 OpenAI Gym (Brockman 等人, 2016) 上达到最先进水平, 并在 Franka-Emika-Panda 机器人上验证了实时控制 ($> 120\text{Hz}$)。

2. 相关工作

本文根据引言中确定的三个类别对相关工作进行了梳理。

2.1. 多步生成策略

扩散策略 (Diffusion Policy, Chi 等人, 2023) 开创了通过去噪扩散进行视觉运动控制的方法, 其中 DP3 (Ze 等人, 2024)

扩展到三维输入。流匹配 (Flow Matching) (Lipman 等人, 2023; Boffi 等人, 2025) 和修正流 (Rectified Flow) (Liu 等人, 2023) 通过学习的速度场提供了替代方案, 应用于流策略 (Flow Policy) (Zhang 等人, 2025a)、捷径流 (shortcut flows) (Chen 等人, 2025c) 以及基础模型 π_0 (团队, 2024)、 $\pi_{0.5}$ (团队, 2025)。尽管具有表现力, 这些方法每个动作需要数十到数百次函数评估 (NFE), 并且仅通过模仿学习无法超越专家性能。

2.2 一步生成方法

一致性策略 (Consistency Policy) (Prasad 等, 2024) 与 1-DP (Wang 等, 2025) 通过蒸馏将生成过程缩减为一步, 但仍受限于教师模型的能力 (Lu 等, 2024; Yan 等, 2025; Clemente 等, 2025; Chen 等, 2025a; Liu 等, 2025a)。MP1 (Sheng 等, 2026) 首次将平均流 (MeanFlow) (Geng 等, 2025a; b; Guo 等, 2025; Hu 等, 2025) 应用于无蒸馏的一步生成, 证明了平均速度流模型可以在没有扩散教师模型的情况下部署。然而, 一步生成往往会导致中间表征的坍塌。尽管分散正则化已被应用于多步扩散模型 (Wang & He, 2025; Zou 等, 2026) 以提升表征质量, 但其在无蒸馏一步策略中的应用尚未被探索。本文首次研究了将分散正则化应用于平均流以实现稳定的一步生成。

2.3. 生成策略的强化学习微调

DPPO (Ren 等, 2025) 开创了扩散策略的强化学习微调 (Schulman 等, 2017), ReinFlow (Zhang 等, 2025b) 将其扩展到基于流的策略, π_{RL} (Chen 等, 2025b) 进一步将强化学习扩展到视觉语言动作模型 (VLA) (Lu 等, 2025; Liu 等, 2025b; McAllister 等, 2025)。然而, 这些方法均基于多步策略, 继承了计算开销 (Zou 等, 2026; Wagenmaker 等, 2025)。此外, 强化学习微调可能不稳定, 性能会随着训练进行而下降。行为克隆正则化 (Nair 等, 2021) 通过防止灾难性遗忘来缓解这一问题。DMPO 通过为真正的一步策略开发稳定的强化学习微调来填补这一空白。

3. 方法论

本节介绍 DMPO 方法论。我们首先在 3.1 节给出框架概述, 然后详细阐述我们的三项贡献: 3.2 节的平均流一步生成、3.3 节的分散正则化以及 3.4 节的带行为克隆正则化的强化学习微调。详细的数学推导见附录 C。图 2 展示了完整的 DMPO 流程, 分为两个阶段。我们的核心设计理念是通过结合轻量级模型架构与互补的算法设计, 实现真正的实时机器人控制。

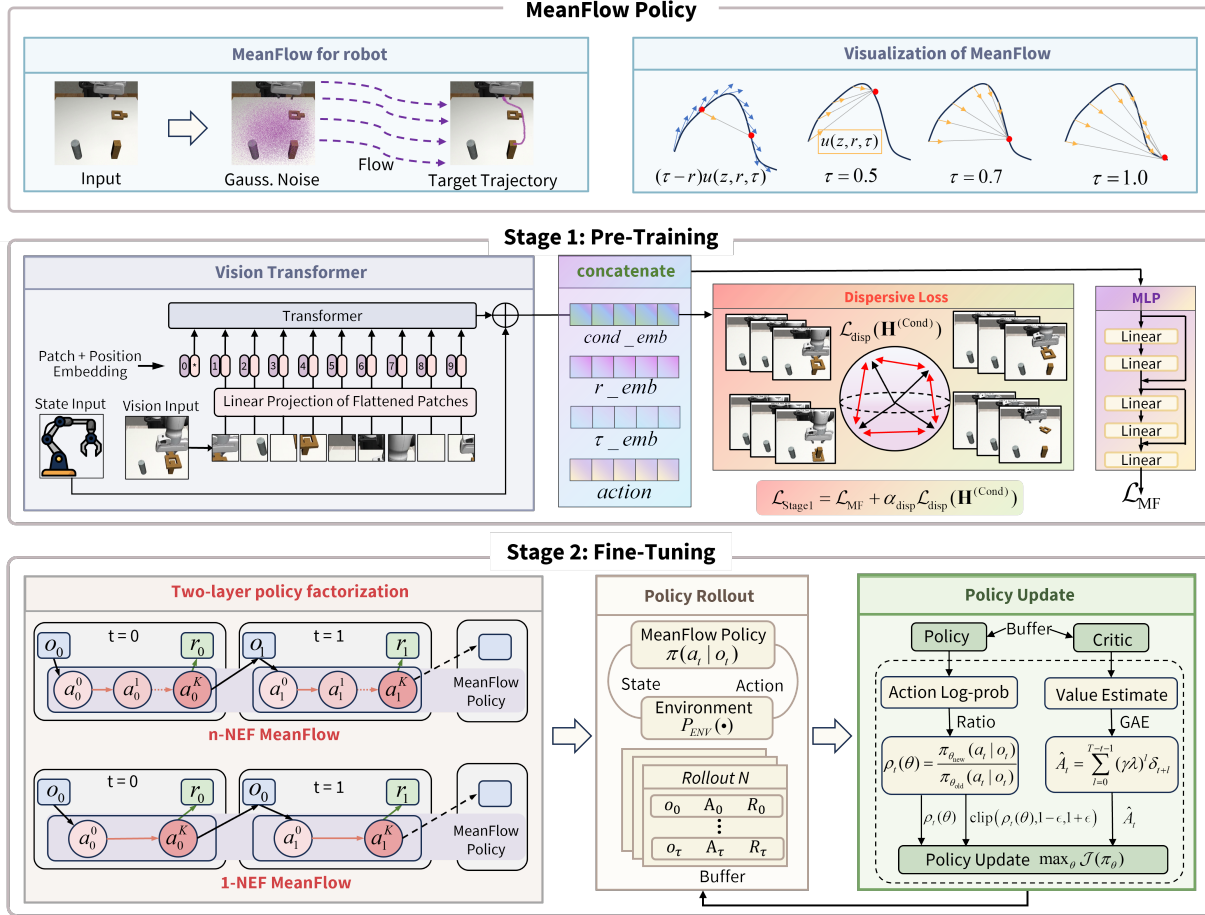


图 2. DMPO 框架概览。阶段 1（上、中）：使用分散平均流进行预训练。平均流通过视觉变换器编码学习将噪声转化为动作的速度场，并采用分散损失防止表示坍塌。阶段 2（下）：将 PPO 微调形式化为两层策略分解：外层为真实环境马尔可夫决策过程，内层潜在链仅对动作分布进行重参数化。

3.1. 框架概览

面向实时控制的轻量级架构。遵循先前机器人学习工作（Peroldi 等，2021），我们采用单层轻量级视觉变换器，具有 128 维嵌入，并结合多层感知机动作头，总参数量仅为 1.78M（比 ViT-Base 小 $48 \times$ 倍）。该紧凑设计使推理速度超过 120Hz，满足实时控制需求。

阶段 1：预训练分散平均流策略优化（DMPO）。顶部和中部面板展示了预训练过程。平均流学习速度场 $u(z, s, \tau)$ ，将高斯噪声转化为动作轨迹，支持少步或单步采样。视觉输入由视觉变换器处理，并与机器人状态融合为条件嵌入。多层感知机处理该嵌入及时间嵌入以预测速度场。对条件嵌入施加分散损失以防止表示坍塌。

Stage 2: RL Fine-Tuning DMPO. The bottom panel shows

微调阶段。我们采用两层策略分解：外层为具有奖励和访问频率的真实环境马尔可夫决策过程，内层包含潜在去噪链 $a^{K-1} \rightarrow \dots \rightarrow a^0$ ，仅进行重参数化 $\pi_\theta(a | o)$ 。由于潜在状态既不与环境交互也不产生奖励，PPO 优势和价值目标始终定义在外层环境上。策略更新遵循带裁剪概率比 $\rho_t(\theta)$ 和广义优势估计优势 $\hat{A}_t$ 的 PPO，同时行为克隆正则化防止灾难性遗忘。

3.2. DMPO 中的单步生成策略

MeanFlow 通过学习平均速度场而非瞬时速度来扩展流匹配。给定目标动作 a （在 $\tau=0$ 处）与噪声 $\epsilon \sim \mathcal{N}(0, I)$ （在 $\tau=1$ 处）之间的线性插值 $z_\tau = (1 - \tau)a + \tau\epsilon$ ，区间 $[r, \tau]$ 上的平均速度为：

$$u(z_\tau, r, \tau) \triangleq \frac{1}{\tau - r} \int_r^\tau v(z_\xi, \xi) d\xi, \quad (1)$$

满足位移恒等式 $(\tau - r)u(z_\tau, r, \tau) = 0$ 。对其求导 $z_\tau - z_r$ 得到 MeanFlow 恒等式：

$$u(z_\tau, r, \tau) = v(z_\tau, \tau) - (\tau - r) \frac{d}{d\tau} u(z_\tau, r, \tau), \quad (2)$$

其中全导数 $\frac{d}{d\tau} u = v_\tau \cdot \nabla_z u + \partial_\tau u$ 涉及雅可比向量积 (JVP, 见附录 C.1 中的公式 20)。MeanFlow 训练目标为：

$$\mathcal{L}_{\text{MF}} = \mathbb{E}_{(o, a), \epsilon, (r, \tau)} \left[\|u_\theta(z_\tau, r, \tau, o) - u_{\text{tgt}}\|^2 \right], \quad (3)$$

$u_{\text{tgt}} = v_\tau - (\tau - r)$ 其中 $\frac{d}{d\tau} u_\theta$ 为目标速度 (公式 21), $v_\tau = \epsilon - a$ 为从数据指向噪声的瞬时速度。包含 JVP 计算和时间采样策略的完整公式见附录 C.1。

3.3. 用于防止坍缩的分散正则化

在一步生成策略中

MeanFlow 存在表示坍缩问题，即不同观测映射到相似嵌入。本文识别出两个促成因素：(1) 平均速度训练目标对细粒度表示差异不够敏感；(2) 少步推理通过提供有限的表示查询机会放大了这一问题。详细的理论分析见附录 D.1。分散正则化通过鼓励中间特征

$\mathbf{H} = \{\mathbf{h}_1, \dots, \mathbf{h}_B\} \subset \mathbb{R}^d$ 在特征空间中分散来从根本上解决该问题。本文探索了四种形式：(1) InfoNCE-L2 通过对比学习最大化成对 L2 距离：

$$\mathcal{L}_{\text{InfoNCE-L2}} = -\mathbb{E} \left[\log \frac{\exp(\|\mathbf{h}_i\|_2^2 / \tau)}{\sum_{k \neq i} \exp(-\|\mathbf{h}_i - \mathbf{h}_k\|_2^2 / \tau)} \right] \quad (4)$$

其中 $\tau > 0$ 为温度参数。本文还探索了

(2) 信息噪声对比估计-余弦 (InfoNCE-Cosine) 用于角度分离, (3) 铰链损失 (Hinge Loss) 用于最小分离间隔, 以及 (4) 基于协方差的损失用于特征去相关。详细公式见附录 C.2。

第一阶段训练目标结合了平均流 (MeanFlow) 和分散正则化：

$$\mathcal{L}_{\text{Stage1}} = \mathcal{L}_{\text{MF}} + \alpha_{\text{disp}} \mathcal{L}_{\text{disp}}(\mathbf{H}^{(\text{Cond})}), \quad (5)$$

其中 $\alpha_{\text{disp}} > 0$ 是平衡系数，我们对视觉变换器编码器输出的条件嵌入 $\mathbf{H}^{(\text{Cond})}$ 应用分散正则化。条件嵌入编码了观测信息 (视觉特征和机器人状态)，是区分不同样本的关键表示

。我们的分析表明，对条件嵌入的分散正则化有效缓解了少步推理中的表示坍缩。详细讨论见附录 D.1。

3.4. 用于一步生成策略的强化学习微调

纯模仿学习无法超越专家演示。我们通过带行为克隆 (BC) 正则化的近端策略优化 (PPO) 微调来解决这一问题。

随机采样。在微调过程中，本文通过引入探索噪声来增强确定性均值流 (MeanFlow)。K 步马尔可夫链中的每个转移遵循：

$$a^{k+1} \sim \mathcal{N}(a^k - \Delta\tau_k \cdot u_\theta(a^k, \tau_{k+1}, \tau_k, o), \sigma^2 I) \quad (6)$$

其中 $\tau_k = 1 - k/K$ 为第 k 步的流动时间， $\Delta\tau_k = \tau_k - \tau_{k+1} = 1/K$ 为步长。

对数概率。对于近端策略优化 (PPO)，联合对数概率在去噪链上分解：

$$\log \pi_\theta(a^{0:K} | o) = \log p_0(a^0) + \sum_{k=0}^{K-1} \log \mathcal{N}(a^{k+1} | \mu_k, \sigma^2 I) \quad (7)$$

其中 $\mu_k = a^k - \Delta\tau_k \cdot u_\theta(a^k, \tau_{k+1}, \tau_k, o)$ 。完整推导过程详细推导见附录 C.3 (公式 31)。

训练目标。我们采用标准近端策略优化 (PPO)，结合截断策略梯度、价值损失和熵奖励 (Schulman 等, 2017)，并加入行为克隆正则化以防止灾难性遗忘：

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_o \left[\|a_\omega(o) - a_\theta(o)\|_2^2 \right] \quad (8)$$

其中 a_ω 是来自参数为 ω 的冻结预训练策略的动作。微调总目标为：

$$\mathcal{L}_{\text{Stage2}} = \mathcal{L}_{\text{PPO}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} \quad (9)$$

其中 λ_{BC} 控制正则化强度 (完整公式见式 39)。详细算法见附录 C.3。

4. 实验

我们在四个 RoboMimic 操作任务 (Mandlekar 等, 2021) 和 OpenAI Gym 运动任务 (Brockman 等, 2016) 上评估预训练和微调，每个任务使用 100 个评估回合。为确保与基线算法 ReinFlow (Zhang 等, 2025b) 和 DPPO (Ren 等, 2025) 的公平比较，我们采用与其实验相同的简化 RoboMimic 数据集。详细内容见附录 B。

我们的实验旨在回答与本文贡献相对应的三个研究问题：

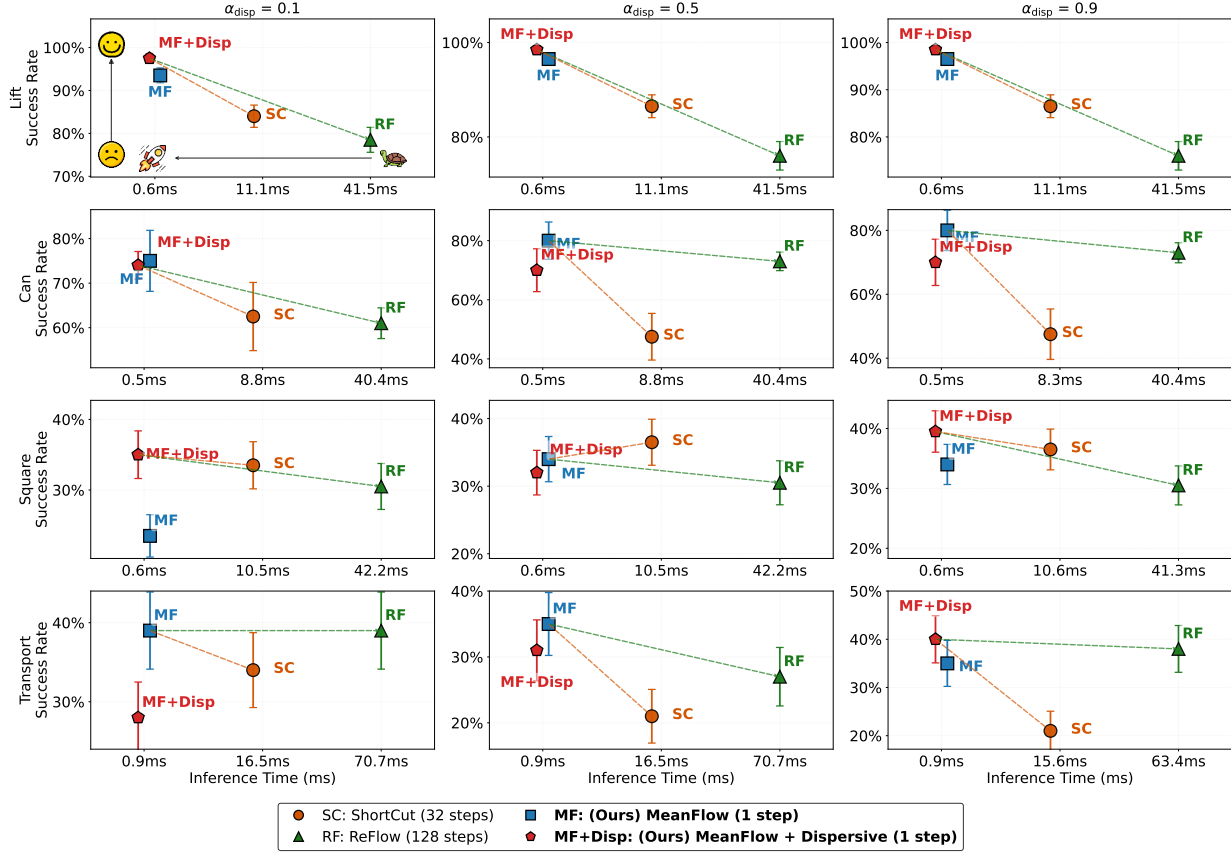


图 3. 阶段 1（预训练）：在四个 RoboMimic 任务和三种分散权重 ($\alpha_{\text{disp}} \in \{0.1, 0.5, 0.9\}$) 下，推理效率与成功率的权衡。左上区域（快速 + 高成功率）为理想区域。MF 和 MF+Disp 位于帕累托前沿，相比 ShortCut 实现 $6 - 10 \times$ 倍加速，相比 ReFlow 实现 $25 - 40 \times$ 倍加速，同时保持更优的成功率。

研究问题 1：一步生成能否在实现更快推理的同时，达到或超越多步扩散策略？

研究问题二：色散正则化对于防止一步生成中的模式崩塌是否必不可少？

研究问题三：在线强化学习微调能否突破离线专家数据的性能上限？

4.1. 阶段一：DMPO 的预训练结果（研究问题一与研究问题二）

4.1.1. ROBOMIMIC 预训练任务

图 3 表明，平均流（MeanFlow）通过真正的一步生成实现了显著的推理效率提升。对比在英伟达 RTX 4090 图形处理器上测得的单样本推理时间：一步的 MF/MF+分散正则化仅需 $\sim 0.6 - 1.0$ 毫秒，而 32 步的捷径方法（ShortCut）需 $\sim 9 - 16$ 毫秒，128 步的重流方法（ReFlow）需 $\sim 40 - 70$ 毫秒。本文方法比捷径方法快 $15 - 20 \times$ 倍，比重流方法快 $60 \times$ 倍。

更重要的是，MF 和 MF+分散正则化占据了帕累托前沿，对应图 3 中的左上区域：

尽管仅使用单步去噪，本文方法仍能达到与需要数量级更多计算量的基线方法相当或更优的成功率。

以 $\alpha_{\text{disp}} = 0.1$ 为例：在提升任务上，MF+分散正则化实现了接近 100% 的成功率。在抓取任务上，MF 和 MF+分散正则化均显著优于捷径方法和重流方法。在搬运任务上，MF 保持强劲性能，同时比重流方法快 $60 \times$ 倍。

图 4 揭示了分散正则化在不同去噪步数下的效果。以绿色三角形表示的重流方法在低步数下表现不佳：在提升任务上，1 步成功率仅为 $\sim 60\%$ ，需要 32 - 64 步才能接近 95%。这表明学习到的向量场在少步采样下产生高度弯曲的轨迹，导致较大的积分误差。以橙色圆圈表示、专为少步推理设计的捷径方法比重流方法有所改进，但性能不稳定且上限有限。以蓝色方块表示的 MF 即使没有分散正则化，也能在 1 - 5 步内达到接近饱和的性能。以红色圆圈表示的 MF+分散正则化进一步稳定了性能，在较难任务（如方形和搬运）上，阴影区域更窄，表明方差明显降低。

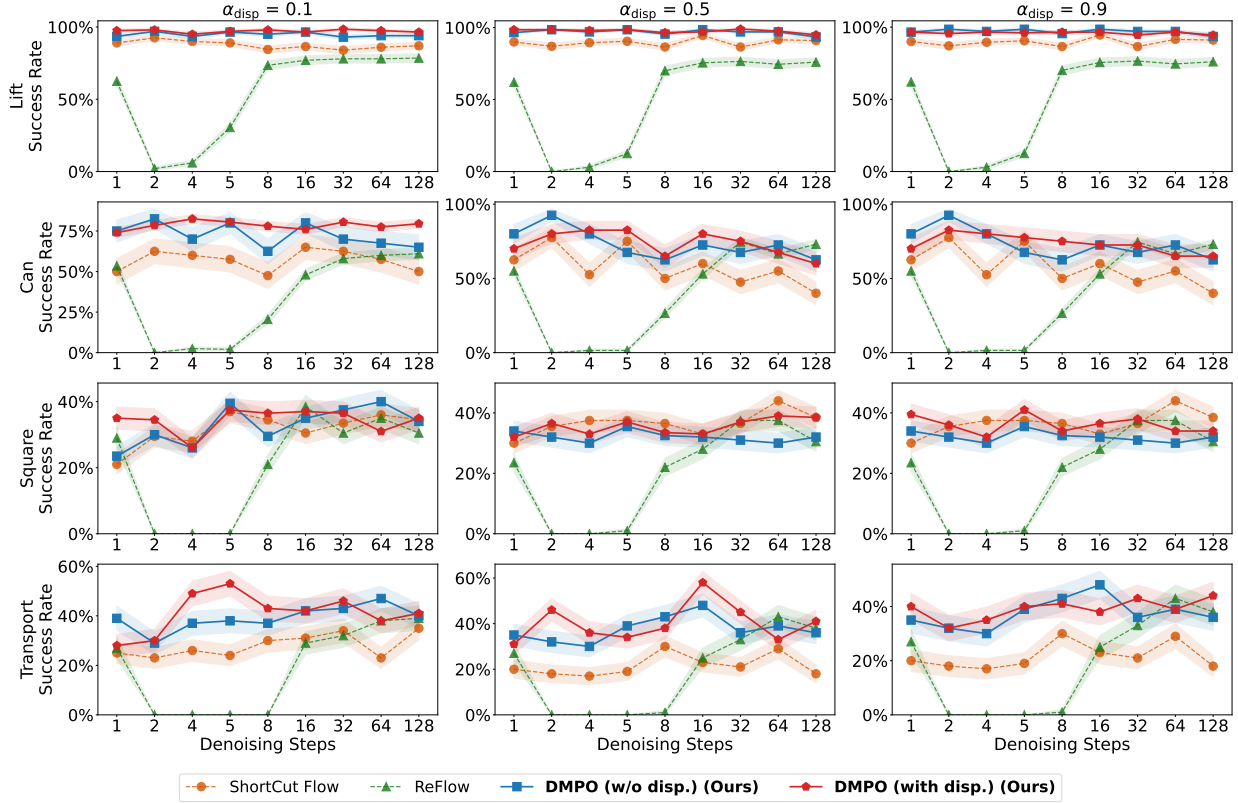


图 4. 阶段 1 (预训练)：在 4×3 网格上成功率与去噪步数的关系。行对应难度递增的提升、抓取、方形和搬运任务，列对应 $\alpha_{\text{disp}} \in \{0.1, 0.5, 0.9\}$ 。平均流变体 MF 和 MF+分散正则化在 1-5 步内达到接近饱和的性能，而重流方法和捷径方法需要 32-128 步。分散正则化降低了复杂任务上的方差。

三列揭示了 α_{disp} 的效果：在所有测试值 (0.1、0.5、0.9) 下，分散正则化均一致地提高了任务成功率，表明其具有鲁棒性而非超参数敏感性。在复杂任务上改进更为显著，证实了分散正则化能够防止表示坍塌，这对于稳定的一步生成至关重要。

我们观察到成功率提升与任务复杂度之间存在强线性关系 (皮尔逊相关系数 $r = 0.924$)，这给出了一条简单准则：对于更具挑战性的任务应增大 α_{disp} 。细节见附录 A.2.2。

4.1.2. 健身房预训练任务

健身房运动与厨房任务的预训练结果见附录。使用与 ReinFlow 相同的 D4RL 数据集，DMPO 在 Hopper、Walker2d 和 Kitchen 任务上取得了有竞争力或更优的性能。值得注意的是，DMPO 在不同推理步数下保持稳定性能，在 1-4 步区间优势明显：在 Hopper 上，DMPO 取得了最佳的 1 步奖励 1364.1 和 2 步奖励 1484.8。在 Kitchen-Complete 上，DMPO 在 1 步时达到 52.5%，而 ShortCut 为 35%。这些结果

证实 MeanFlow 的少步采样优势不仅限于 RoboMimic，还推广到运动与厨房操作领域，同时分散正则化继续在不同步数下稳定性能。我们还报告了 Humanoid 预训练结果，并指出 Ant 性能差距部分源于 ant-medium-expert-v2 与 v0 之间的数据集版本差异，讨论见附录 A.2.1。

4.2. 第二阶段：DMPO 微调结果 (研究问题 3)

为评估 MeanFlow 策略的 PPO 微调有效性 (贡献 3)，我们在 RoboMimic 操作任务和 OpenAI Gym 运动基准上进行了实验。

4.2.1. ROBOMIMIC 微调结果

图 5 展示了三个 RoboMimic 操作任务 (Can、Square 和 Transport) 的 PPO 微调结果。我们省略了 Lift，因为 MeanFlow 在第一阶段预训练中已取得接近 100% 的成功率，没有通过微调改进的空间。DMPO 取得了与所有基线相比有竞争力或更优的性能，同时

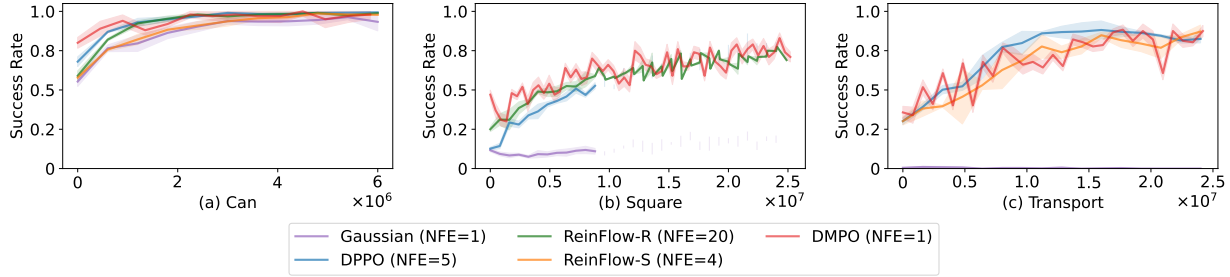


图 5. 第二阶段（近端策略优化微调）：在 RoboMimic 操作任务上的性能。蓝色 MeanFlow 与黄色 DPPO、绿色高斯、红色 ReinFlow-S 和紫色 ReinFlow-R 进行比较。由于 MeanFlow 在第一阶段预训练中达到 100% 成功率，Lift 任务被省略。

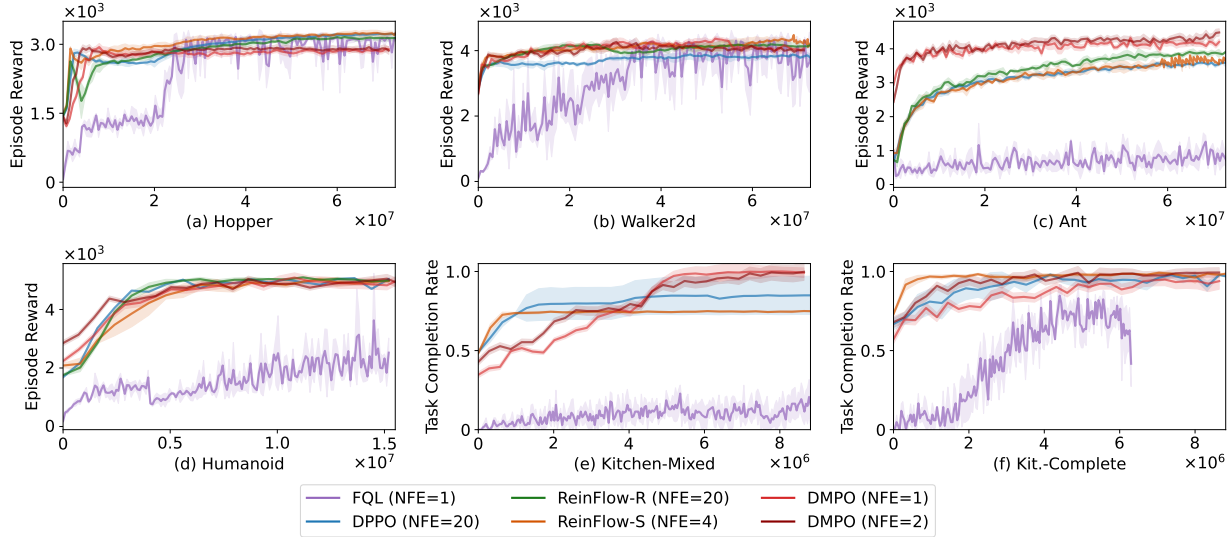


图 6. 第二阶段（近端策略优化微调）：面板 a - d 为 OpenAI Gym 运动任务性能，面板 e - g 为 Kitchen 操作任务性能。蓝色 MeanFlow 与黄色 DPPO、红色 ReinFlow-S、紫色 ReinFlow-R 和浅蓝色 FQL 进行比较。

仅需 1 个去噪步骤。在 Can 任务上，DMPO 比所有基线更快达到 100% 成功率，优于 DPPO 的 99.3% 和 ReinFlow-R 的 99.0%。在 **Square** 任务上，DMPO 达到 83% 的峰值成功率，与 DPPO 的 78.3% 和 ReinFlow-R 的 80% 相当。在 **Transport** 任务上，DMPO 达到 88% 的成功率，与 ReinFlow-S 性能相当，同时显著优于高斯基线。

4.2.2. OpenAI Gym 与 Kitchen 运动任务结果

我们在四个 OpenAI Gym 运动任务（Hopper、Walker2d、Ant 和 Humanoid）以及来自 D4RL（Fu 等人，2020）的两个 Kitchen 任务（Complete 和 Partial）上评估近端策略优化微调。如图 6 所示，DMPO 在所有任务上实现稳定高效的学习，且推理时仅需单个去噪步骤。

在 Gym 任务上，仅使用 1 个去噪步骤的 DMPO 在 Hopper、Walker2d 和 Humanoid 上达到与多步基线相当的性能（图 6a、b、d），同时在 Ant 上显著

优于所有基线（图 6c）。

在需要长时程顺序控制的厨房任务中，DMPO 在厨房完整任务上迅速达到 100% 完成率（图 6e），并在厨房混合任务和厨房部分任务上保持同样强劲且稳定的性能（图 6f - g）。

总体而言，采用单步去噪的 DMPO 达到或超越了多步扩散基线，包括 20 步的 DPPO 和 4 步的 ReinFlow，表明单步生成保持了微调的有效性。使用 $\lambda_{BC} = 0.05 - 0.2$ 的行为克隆正则化防止了灾难性遗忘，并在不同任务复杂度下相对于预训练模仿基线实现了一致的提升。与其他单步生成模型及额外基线的详细比较见附录 A.1。

4.3. 物理机器人实验

为验证实际应用性，我们将 DMPO 部署在一台配备英特尔的 7 自由度 Franka-Emika-Panda 机器人上

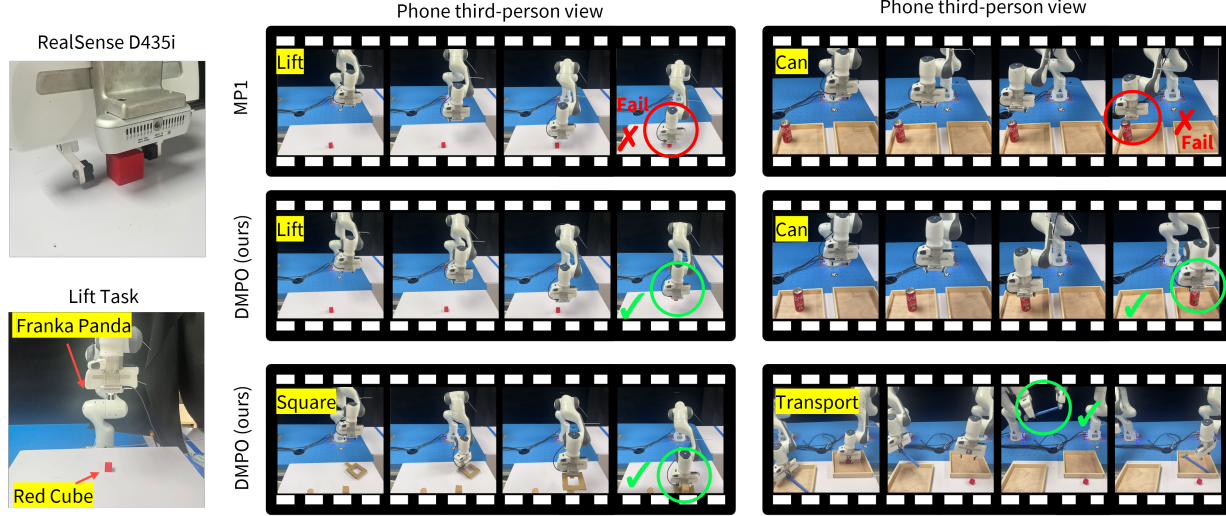


图 7. DMPO 在 Franka Panda 机器人上的真实世界部署。左：硬件设置，使用英特尔实感 D435i 相机进行视觉观测。右：MP1 基线（顶行）与我们的 DMPO 方法（第 2-3 行）在四个操作任务上的定性比较。MP1 在举起和罐装任务上因抓取不精确而失败，而 DMPO 成功完成所有四个任务（举起、罐装、方块、运输），展示了卓越的仿真到现实迁移能力。绿色对勾表示任务成功完成；红色叉号表示失败。

在 RTX 2080 上，采用 1 步推理的 DMPO 仅需 2.6 毫秒进行神经网络前向传播，相比之下，5 步 ShortCut 需要 12.0 毫秒，20 步 ReFlow 需要 47.7 毫秒，分别实现了 $4.6 \times$ 倍和 $18 \times$ 倍的加速。包括所有流水线阶段，DMPO 的总延迟为 9.6 毫秒，可实现超过 100Hz 的控制频率。

如图 7 所示，我们在真实硬件上比较了 MP1 与 DMPO。我们专注于 MP1，因为它是唯一其他基于 MeanFlow 的单步方法；其他基线（DPPO、ReinFlow）在仿真中进行了比较，而在真实机器人上部署多步方法会带来显著的延迟。MP1 在举起和罐装任务上因表示坍塌导致的抓取不精确而失败，而 DMPO 成功完成了包括方块和运输在内的所有四个任务。这些结果表明 DMPO 在仿真中的优势迁移到了物理硬件上，验证了实时控制能力和仿真到现实的泛化能力。详细的设置和延迟分解见附录 A.4。

4.4. 额外分析

我们在附录 A 中提供了额外的分析：与单步生成基线的比较表明，DMPO 在数据效率高的同时实现了优越的性能（附录 A.1），推理时间比较显示相对于基线有显著的加速（附录 A.1.2），整体雷达图比较确认在所有评估维度上均获得最高分（附录 A.1.3）。

5. 结论

我们提出了 DMPO，一个统一的框架，通过真正的单步生成实现实时机器人控制。通过整合平均流（MeanFlow）、分散正则化和强化学习微调，DMPO 同时实现了快速推理和高任务性能，避免了限制现有方法的效率与质量权衡。平均流提供了无需知识蒸馏的数学推导单步推理，而分散正则化通过保持表示多样性确保稳定生成。强化学习微调进一步使策略超越专家演示，突破了性能上限。

DMPO 在 RTX 4090 上仅以 0.6 毫秒的推理延迟实现了真正的实时机器人控制。在 RoboMimic 操作和 OpenAI Gym 运动基准上的实验表明，其性能接近最先进水平。与没有分散正则化的普通平均流相比，我们的分散正则化将任务成功率提高了 5 - 10%。此外，与仅使用行为克隆预训练相比，强化学习微调将性能提升了超过 50%。我们成功地将 DMPO 部署在 Franka-Emika-Panda 机器人上，实现了实时控制并验证了有效的仿真到现实迁移。这些结果表明，DMPO 同时实现了快速推理和高任务性能，证实了我们的核心主张：一步生成足以实现实时生成式机器人控制。

影响声明

本文提出了 DMPO，一个通过高效单步生成策略实现实时机器人控制的框架。通过将推理延迟从数百毫秒降低到几毫秒，我们的工作使得在时间关键型应用中实现更灵敏、更安全的人机交互成为可能。效率的提高还降低了机器人学习的计算需求，可能降低训练机器人系统的环境成本。我们预计这项工作不会产生超出机器人研究常见范围的负面社会影响。

参考文献

- Boffi, N. M., Albergio, M. S., and Vanden-Eijnden, E. Flow map matching. *Transactions on Machine Learning Research (TMLR)*, 2025.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Chen, H., Zhang, K., Tan, H., Guibas, L., Wetzstein, G., and Bi, S. pi-flow: Policy-based few-step generation via imitation distillation. *arXiv preprint arXiv:2510.14974*, 2025a.
- Chen, J. et al. π_{RL} : Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025b.
- Chen, W., Ou, Z., and Li, Y. Neural flow samplers with shortcut models. In *ICLR 2025 Workshop on Foundation Models for Process Intelligence (FPI)*, 2025c.
- Chen, X. and He, K. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. In *Conference on Robot Learning (CoRL)*, 2023.
- Clemente, A. et al. Two-steps diffusion policy for robotic manipulation via genetic denoising. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- Geng, Z., Deng, M., Bai, X., Kolter, J. Z., and He, K. Mean flows for one-step generative modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.
- Geng, Z. et al. Improved mean flows on the challenges of fast-forward generative models. *arXiv preprint arXiv:2512.02012*, 2025b.
- Guo, X. et al. Splitmeanflow: Interval splitting consistency in few-step generative modeling. *arXiv preprint arXiv:2507.16884*, 2025.
- Hu, X. et al. Meanflow transformers with representation autoencoders. *arXiv preprint arXiv:2511.13019*, 2025.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Liu, H., Xie, Q., Ye, T., Deng, Z., Chen, C., Tang, S., Fu, X., Lu, H., and Zha, Z.-J. Scott: Accelerating diffusion models with stochastic consistency distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025a.
- Liu, J., Liu, G., Liang, J., Li, Y., Liu, J., Wang, X., Wan, P., Zhang, D., and Ouyang, W. Flow-grpo: Training flow matching models via online rl. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b.
- Liu, X., Gong, C., and Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.
- Lu, G., Gao, Z., Chen, T., Dai, W., Wang, Z., and Tang, Y. Manim: Real-time 3d diffusion policy via consistency model for robotic manipulation. *arXiv preprint arXiv:2406.01586*, 2024.
- Lu, G., Guo, W., Zhang, C., Zhou, Y., Jiang, H., Gao, Z., Tang, Y., and Wang, Z. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.
- Mandlekar, A., Xu, D., Wong, J., Nasiriany, S., Wang, C., Kulkarni, R., Fei-Fei, L., Savarese, S., Zhu, Y., and Martín-Martín, R. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2021.
- McAllister, D., Ge, S., Yi, B., Kim, C. M., Weber, E., Choi, H., Feng, H., and Kanazawa, A. Flow matching policy gradients. *arXiv preprint arXiv:2507.21053*, 2025.
- Nair, A., Dalal, M., Gupta, A., and Levine, S. Awac: Accelerating online reinforcement learning with offline datasets. In *International Conference on Learning Representations (ICLR)*, 2021. *arXiv:2006.09359*.

- Peroldi, F., Lynch, C., Wahid, A., Tompson, J., Yu, T., Gu, S. S., Hsu, J., Perona, P., Zeng, A., Dwibedi, D., and Levine, S. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2021.
- Prasad, A., Lin, K., Wu, J., Zhou, L., and Bohg, J. Consistency policy: Accelerated visuomotor policies via consistency distillation. In *Robotics: Science and Systems (RSS)*, 2024.
- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., Majumdar, A., Burchfiel, B., Dai, H., and Simchowitz, M. Diffusion policy policy optimization. In *International Conference on Learning Representations (ICLR)*, 2025.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shafiullah, N. M. M., Cui, Z. J., Altanzaya, A., and Pinto, L. Behavior transformers: Cloning k modes with one stone. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Sheng, J., Wang, Z., Li, P., and Liu, M. Mp1: Meanflow tames policy learning in 1-step for robotic manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- Team, P. I. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Team, P. I. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of Machine Learning Research (PMLR)*, 2025.
- Wagenmaker, A., Nakamoto, M., Zhang, Y., Park, S., Yagoub, W., Nagabandi, A., Gupta, A., and Levine, S. Steering your diffusion policy with latent space reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2025.
- Wang, R. and He, K. Diffuse and disperse: Image generation with representation regularization. *arXiv preprint arXiv:2506.09027*, 2025.
- Wang, Z. et al. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. In *International Conference on Machine Learning (ICML)*, 2025.
- Yan, X. et al. Manifold: A general robot manipulation policy via consistency flow training. In *Conference on Robot Learning (CoRL)*, 2025.
- Ze, Y., Zhang, G., Zhang, K., Hu, C., Wang, M., and Xu, H. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. In *Robotics: Science and Systems (RSS)*, 2024.
- Zhang, Q., Liu, Z., Fan, H., Liu, G., Zeng, B., and Liu, S. Flowpolicy: Enabling fast and robust 3d flow-based policy via consistency flow matching for robot manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025a.
- Zhang, T., Wang, M., Yu, W., Choudhury, S., Garg, A., and Zhao, D. Reinfo: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b.
- Zou, G., Lyu, J., Li, X., and Lu, Z. D2ppo: Diffusion policy policy optimization with dispersive loss. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.

附录概述

本附录提供的补充材料按如下结构组织：

附录 A：实验结果提供了全面的实验结果，包括对比评估、在 RoboMimic 和 D4RL 基准上的第一阶段预训练结果、第二阶段微调结果以及实体机器人实验细节。

附录 B：数据集分析与可视化详细分析了 RoboMimic 和 D4RL 数据集的选择依据与分布特征。

附录 C：算法与方法公式包含第一阶段带分散正则化的 MeanFlow 预训练和第二阶段 PPO 微调的完整数学公式。

附录 D：理论分析与统一框架设计为分散正则化提供了信息论基础，并解释了同时考虑模型架构与算法改进的统一框架设计。

A. 实验结果

本附录按训练阶段提供全面的实验结果。A.1 节给出 RoboMimic 上的对比评估与验证，A.2 节给出 Gym 上的第一阶段预训练结果及 RoboMimic 任务复杂度分析，A.3 节给出 Gym 上的第二阶段微调结果，A.4 节详述实体机器人实验。

A.1. 对比评估与验证

A.1.1. 数据集变体间的性能比较

数据集选择。为确保与基线算法 DPPO (Ren 等, 2025) 和 ReinFlow (Zhang 等, 2025b) 进行公平比较，我们采用其实验中使用的同一简化 Robomimic MH 数据集。详细的数据集分析与可视化见附录 B。

表 1. 数据集变体间的性能比较（多人数据）。所有实验均使用多人（mh）数据集变体。完整数据集每个任务包含 300 条轨迹，简化数据集包含 100 条轨迹。

Data	Method	Venue	NFE	Dist.	Lift	Can	Square	Transport
<i>Full Dataset (300 trajectories) - Baselines</i>								
Full	LSTM-GMM	CoRL'21 (Mandlekar et al., 2021)	-	✗	0.93	0.81	0.59	0.20
Full	IBC	CoRL'21 (Peroldi et al., 2021)	-	✗	0.02	0.01	0.00	0.00
Full	BET	NeurIPS'22 (Shafiullah et al., 2022)	-	✗	0.99	0.90	0.43	0.06
Full	DP-C	RSS'23 (Chi et al., 2023)	100	✗	0.97	0.96	0.82	0.46
Full	DP-T	RSS'23 (Chi et al., 2023)	100	✗	1.00	0.94	0.81	0.35
Full	CP	RSS'24 (Prasad et al., 2024)	3	✓	-	-	0.71	0.37
Full	CP	RSS'24 (Prasad et al., 2024)	1	✓	-	-	0.65	0.38
Full	OneDP-D	ICML'25 (Wang et al., 2025)	1	✓	-	-	0.78	0.71
Full	OneDP-S	ICML'25 (Wang et al., 2025)	1	✓	-	-	0.77	0.72
<i>Simplified Dataset (100 trajectories)</i>								
Simplified	DPPO	ICLR'25 (Ren et al., 2025)	20	✗	1.00	1.00	0.78	0.80
Simplified	ReFlow-R	NeurIPS'25 (Zhang et al., 2025b)	20	✗	1.00	0.99	0.80	-
Simplified	ReFlow-S	NeurIPS'25 (Zhang et al., 2025b)	4	✗	1.00	0.99	-	0.85
Simplified	MP1 [†]	AAAI'26 (Sheng et al., 2026)	1	✗	0.95	0.80	0.35	0.38
Simplified	DMPO (Ours)	-	1	✗	1.00	1.00	0.83	0.88

[†]The original MP1 uses 3D point cloud input, which is not directly applicable to RoboMimic's 2D image observations. Our MP1 results are reproduced using MeanFlow with our lightweight architecture and 2D image input for fair comparison. MP1 is the only other MeanFlow-based one-step method, making it the most direct comparison; deploying multi-step methods on real robots incurs significant latency unsuitable for real-time control. Results for DP-C, DP-T, CP, and OneDP are sourced from (Wang et al., 2025).

与一步生成方法的比较。除基线比较外，我们还将与近期最先进的一步生成方法进行比较，包括 OneDP (Wang 等, 2025) 和一致性策略 (Consistency Policy) (Prasad 等, 2024)。这些方法均从多步教师模型蒸馏而来，因此我们也将其教师模型 (DP-C、DP-T) 作为参考。如表 1 所示，DMPO 显著优于现有一步生成方法及其教师模型。该结果与我们的理论分析一致：蒸馏学生模型的性能很少能超越其教师模型。

数据效率。值得注意的是，表 1 表明，仅使用官方数据集 1/3 数据（100 条轨迹对比 300 条轨迹）的 DMPO 即可达到甚至超越在完整数据集上训练的一步生成方法。这一优势在复杂任务（如 Transport）上尤为突出，DMPO 的成功率达到 88%，而 OneDP-S 为 72%，凸显了本算法卓越的数据利用效率。

关于基线的说明。完整数据集上一步生成基线 (DP-C、DP-T、CP、OneDP) 的结果来源于 (Wang 等, 2025)。

与 MP1 的比较。MP1 (Sheng 等, 2026) 是一项同期工作，同样探索了基于均值预测的一步生成。如表 1 所示，虽然 MP1 在较简单任务上取得了合理性能，在 Lift 上达到 95%，在 Can 上达到 80%，但在复杂操作任务上表现显著不佳，在 Square 上仅为 35%，在 Transport 上仅为 38%。相比之下，DMPO 在所有任务上均大幅超越 MP1，在 Lift 和 Can 上实现了完美的 100% 成功率，同时将 Square 从 35% 显著提升至 83%，将 Transport 从 38% 显著提升至 88%。这一性能差距凸显了本文所提出的分散正则化在防止表示坍塌方面的重要性，使 DMPO 即使在复杂的多模态动作分布下也能保持高质量生成。

A.1.2. 模型架构与效率比较

表 2 比较了不同策略学习方法的模型架构与计算效率。前六列描述了模型配置：视觉编码器类型、动作网络架构、参数量、检查点大小和输入模态。后六列报告了在 RTX 4090 和 RTX 2080 GPU 上的推理性能，包括推理时间、控制频率以及相对于使用 100 步 DDPM 采样的扩散策略的加速比。

表 2. 模型架构与效率比较。不同策略学习方法的模型配置与推理性能比较：DP（扩散策略 (Chi 等, 2023)）、CP（一致性策略 (Prasad 等, 2024)）、ReFlow 和 ShortCut (Zhang 等, 2025b) 以及 MP1 (Sheng 等, 2026)。加速比相对于 DP-DDPM (100 步) 计算。所有测量均在批大小为 1 的条件下进行。注：OneDP (Wang 等, 2025) 和 2-DP (Clemente 等, 2025) 因代码未公开，无法进行准确的推理时间测量，故未纳入。

Model	Vision	Action	Params	Size	Steps	NVIDIA RTX 4090			NVIDIA RTX 2080		
						Time	Freq	Speedup	Time	Freq	Speedup
DP	ResNet-18×2	UNet	281.19M	~4.4GB	100 (DDPM)	391.1ms	2.6Hz	1×	2007.5ms	0.5Hz	1×
					16 (DDIM)	63.7ms	15.7Hz	6×	385.3ms	2.6Hz	5×
					10 (DDIM)	40.3ms	24.8Hz	10×	220.4ms	4.5Hz	9×
CP	ResNet-18×2	UNet	284.86M	~4.4GB	1	5.4ms	187Hz	73×	35.5ms	28Hz	56×
ReFlow	light ViT	MLP	1.78M	~28MB	20	8.4ms	119Hz	46×	47.7ms	21Hz	42×
ShortCut	light ViT	MLP	1.78M	~28MB	5	2.5ms	396Hz	155×	12.0ms	83Hz	167×
MP1	PointNet	UNet	255.80M	~4GB	1	4.1ms	244Hz	96×	21.4ms	47Hz	94×
					2	7.7ms	130Hz	51×	41.6ms	24Hz	48×
					5	18.6ms	54Hz	21×	102.1ms	10Hz	20×
DMPO (Ours)	light ViT	MLP	1.78M	~28MB	1	0.6ms	1770Hz	694×	2.6ms	385Hz	772×
					2	1.1ms	946Hz	371×	4.9ms	204Hz	410×
					5	2.6ms	386Hz	151×	12.0ms	83Hz	167×

表 2 比较了不同方法的模型配置与推理性能。前五列描述了模型配置：视觉编码器类型、动作网络架构、参数量和检查点大小。后六列报告了在 RTX 4090（用于仿真）和 RTX 2080（用于物理机器人部署）上的推理性能，包括推理时间、控制频率以及相对于 100 步 DP-DDPM 的加速比。

结果表明，DMPO 的快速推理速度由两个因素决定。首先，轻量级模型架构。基于 UNet 的方法如 DP、CP 和 MP1 包含 255 – 285M 参数，而轻量级 ViT+MLP 方法如

ReFlow、ShortCut 和 DMPO 仅使用 1.78M 参数。在 RTX 4090 上比较 CP 与 DMPO 的单步推理，CP 需要 5.4ms，而 DMPO 仅需 0.6ms，表明轻量级架构提供了约 $10 \times$ 的加速。其次，推理步数的减少。比较 10 步的 DP-DDIM 与 1 步的 DMPO，推理时间从 40.3ms 降至 2.6ms，表明一步生成提供了约 $10 \times$ 的额外加速。这两个因素的结合使 DMPO 在 RTX 4090 上实现了 $694 \times$ 的加速，在 RTX 2080 上实现了 $772 \times$ 的加速，分别达到 1770 Hz 和 385 Hz 的控制频率。紧凑的 28MB 模型大小也便于在资源受限的机器人平台上部署。

A.1.3. 整体雷达比较

图 8 展示了使用雷达图在八个评估维度上的定性比较：推理速度、模型轻量性、成功率、数据效率、表示质量、免蒸馏、超越演示和训练稳定性。每个维度按 1 – 5 分制评分，5 分表示性能优异。我们将 DMPO 与 RL 微调方法（包括 ReinFlow (Zhang et al., 2025b)、DPPO (Ren et al., 2025) 和 π_{RL} (Chen et al., 2025b)) 以及生成方法（包括扩散策略 (DP) (Chi et al., 2023)、流策略 (FP) (Zhang et al., 2025a)、1-DP (Wang et al., 2025)、MP1 (Sheng et al., 2026) 和一致性策略 (CP) (Prasad et al., 2024)) 进行比较。

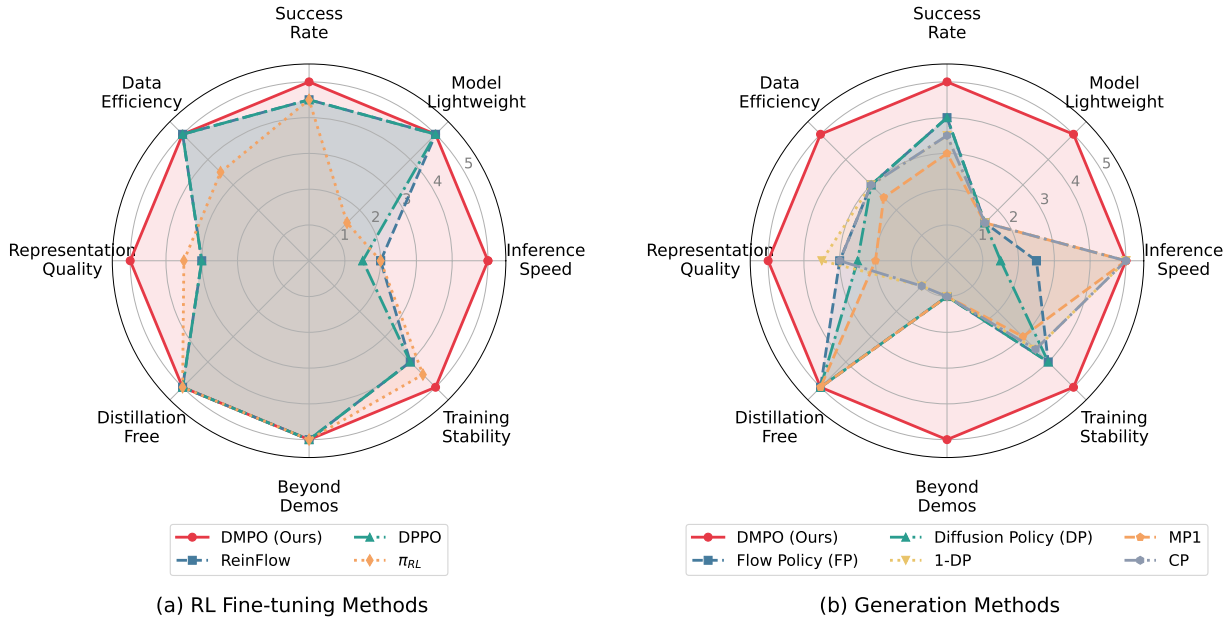


图 8. 八个维度的整体雷达比较。(a) RL 微调方法：DMPO 形成外包络，在所有维度上取得最高分。虽然 ReinFlow 和 DPPO 与 DMPO 共享相同的轻量级架构和数据效率，但它们需要多步推理，且表示质量较低。(b) 生成方法：DMPO 通过结合一步推理与轻量级架构、高数据效率以及通过 RL 微调超越演示的能力，优于所有基线。

RL 微调方法（面板 a）。 所有强化学习方法均具备超越演示且无需蒸馏的能力。ReinFlow 与 DPPO 采用与 DMPO 相同的轻量级 ViT+MLP 架构，数据效率相当，但分别受限于 20 步及以上的多步采样需求。 π_{RL} 方法使用更重的模型架构，数据效率较低。只有 DMPO 通过轻量级架构与由色散 MeanFlow 公式实现的一步推理能力相结合，在全部八个维度上取得最高分。

生成方法（面板 b）。 比较揭示了现有方法之间的明显权衡。DP 和 FP 等多步基线方法在训练稳定时取得中等成功率，但推理速度慢且模型架构沉重。1-DP 和 CP 等蒸馏一步方法加速了推理，但依赖教师模型且无法超越演示性能。MP1 等无教师一步方法绕过了蒸馏，但存在严重的表示坍塌且使用沉重架构。DMPO 是唯一在全部八个维度上达到顶级性能的方法，表明本文方法通过轻量级架构与先进算法设计的结合，成功解决了效率-稳定性-适应性三难困境。

A.2. 阶段 1：预训练结果

A.2.1. 阶段 1：健身房预训练结果

表格概览。表 3 展示了在 OpenAI Gym 运动任务（包括 Hopper、Walker2d、Ant 和 Humanoid）以及 Franka Kitchen 操作任务上的预训练结果。对于运动任务，我们报告回合奖励；对于 Kitchen 任务，我们报告任务完成率。该表将 DMPO 与 ReinFlow 论文（Zhang 等人，2025b）中的两个基线方法进行了比较：ReFlow 使用标准修正流，ShortCut 使用基于回流（reflow）的捷径模型。所有基线结果均直接取自 ReinFlow 论文的开源数据，确保在相同评估协议下进行公平比较。每一行展示了一种方法在推理步数从 1 到 64 步变化时的性能，粗体值表示每个步数下的最佳结果。“Impr.”列显示了 DMPO 相对于 ShortCut 的相对改进，绿色表示改进，红色表示退化。我们包含两个 DMPO 变体用于消融：黄色高亮行表示没有分散正则化的 DMPO，蓝色高亮行表示我们的完整方法。

表 3. 阶段 1（预训练）：DMPO 与 ReinFlow（ReFlow 和 ShortCut）的性能比较。运动任务报告回合奖励，Kitchen 任务报告任务完成率。每个任务的最佳结果以粗体显示。绿色 / 红色 Impr.列显示在每个去噪步骤上相对于 ReFlow 的相对改进。

Task	Method	1	Impr.	2	Impr.	4	Impr.	8	Impr.	16	Impr.	32	Impr.	64	Impr.
<i>Locomotion Tasks (Episode Reward)</i>															
Hopper	ReFlow	1192.6	-	932.4	-	1592.7	-	1584.3	-	1479.1	-	1451.1	-	1389.3	-
	ShortCut	1095.3	-8%	1253.0	+34%	1400.1	-12%	1417.9	-11%	1458.3	-1%	1389.3	-4%	1395.5	0%
	DMPO w/o disp.	1332.7	+12%	1417.6	+52%	1465.2	-8%	1453.0	-8%	1440.9	-3%	1402.9	-3%	1393.5	0%
	DMPO (Ours)	1364.1	+14%	1484.8	+59%	1520.3	-5%	1538.5	-3%	1503.9	+2%	1414.2	-3%	1429.1	+3%
Walker2d	ReFlow	2805.8	-	2810.2	-	2508.4	-	2895.9	-	2944.3	-	2839.7	-	2728.1	-
	ShortCut	2846.5	+1%	2847.4	+1%	2987.3	+19%	2732.5	-6%	2622.2	-11%	2667.8	-6%	2441.4	-11%
	DMPO w/o disp.	2509.8	-11%	2619.0	-7%	2913.9	+16%	2997.6	+4%	2959.7	+1%	2860.3	+1%	3154.6	+16%
	DMPO (Ours)	2606.0	-7%	3033.0	+8%	2777.2	+11%	2831.3	-2%	2953.5	0%	3044.8	+7%	2954.4	+8%
Ant [†]	ReFlow	894.9	-	-40.3	-	495.8	-	654.4	-	709.9	-	761.3	-	771.4	-
	ShortCut	847.4	-5%	779.4	-	685.0	+38%	642.3	-2%	556.2	-22%	544.4	-28%	511.1	-34%
	DMPO w/o disp.	2924.6	+227%	2804.0	-	3355.1	+577%	3302.8	+405%	3427.2	+383%	3434.7	+351%	3371.6	+337%
	DMPO (Ours)	2929.8	+227%	2906.6	-	3289.7	+564%	3377.5	+416%	3352.7	+372%	3267.2	+329%	3536.1	+358%
Humanoid	ReFlow	1655.1	-	1256.7	-	1765.7	-	2106.2	-	2196.2	-	2157.9	-	2156.6	-
	ShortCut	1457.2	-12%	2394.5	+91%	2371.0	+34%	2096.1	0%	2051.0	-7%	1987.9	-8%	2128.5	-1%
	DMPO w/o disp.	2671.6	+61%	3044.0	+142%	3410.4	+93%	3435.5	+63%	3265.9	+49%	2986.2	+38%	2997.0	+39%
	DMPO (Ours)	2588.7	+56%	2809.8	+124%	3070.5	+74%	3235.6	+54%	2928.0	+33%	2823.5	+31%	3097.5	+44%
<i>Kitchen Tasks (Task Completion Rate, %)</i>															
Kitchen-Complete	ReFlow	0%	-	2.5%	-	20.0%	-	50.0%	-	70.0%	-	75.0%	-	77.5%	-
	ShortCut	35.0%	-	42.5%	+1600%	70.0%	+250%	75.0%	+50%	75.0%	+7%	75.0%	0%	77.5%	0%
	DMPO w/o disp.	55.0%	-	72.5%	+2800%	72.5%	+263%	67.5%	+35%	70.0%	0%	67.5%	-10%	65.0%	-16%
	DMPO (Ours)	52.5%	-	70.0%	+2700%	75.0%	+275%	70.0%	+40%	70.0%	0%	70.0%	-7%	70.0%	-10%
Kitchen-Partial	ReFlow	0%	-	5.0%	-	20.0%	-	37.5%	-	35.0%	-	37.5%	-	35.0%	-
	ShortCut	37.5%	-	37.5%	+650%	37.5%	+88%	37.5%	0%	35.0%	0%	35.0%	-7%	37.5%	+7%
	DMPO w/o disp.	35.0%	-	35.0%	+600%	37.5%	+88%	35.0%	-7%	35.0%	0%	32.5%	-13%	35.0%	0%
	DMPO (Ours)	37.5%	-	42.5%	+750%	42.5%	+113%	42.5%	+13%	42.5%	+21%	42.5%	+13%	45.0%	+29%
Kitchen-Mixed	ReFlow	0%	-	2.5%	-	20.0%	-	40.0%	-	42.5%	-	47.5%	-	45.0%	-
	ShortCut	27.5%	-	40.0%	+1500%	42.5%	+113%	40.0%	0%	40.0%	-6%	40.0%	-16%	37.5%	-17%
	DMPO w/o disp.	25.0%	-	37.5%	+1400%	42.5%	+113%	45.0%	+13%	42.5%	0%	42.5%	-11%	42.5%	-6%
	DMPO (Ours)	27.5%	-	42.5%	+1600%	47.5%	+138%	47.5%	+19%	45.0%	+6%	47.5%	0%	47.5%	+6%

[†] 不同的数据集版本：DMPO 使用 ant-medium-expert-v2，ReinFlow 使用 ant-medium-expert-v0。

主要发现。结果证明了两个主要结论：（1）DMPO 优于基线方法，特别是在少步推理场景中。在大多数任务上，DMPO 在 1-2 步时达到最佳或相当的性能，并在更高步数下保持优越的结果。改进在 Humanoid 和 Kitchen-Complete 任务上尤为显著，DMPO 相对于 ReFlow 和 ShortCut 都显示出大幅提升。（2）分散正则化提高了性能。

比较 DMPO（我们的方法）与没有分散正则化的 DMPO，完整方法始终取得更好的结果，证实分散正则化防止了表示坍塌并实现了更稳定的生成质量。注意，Ant 任务由于数据集版本不同而显示出较大的性能差距（见表脚注）。

A.2.2. 阶段 1：RoboMimic 任务的任务复杂度分析

表格和图示概览。 表 4 展示了在三种分散权重配置（ $\alpha_{\text{disp}} = 0.1, 0.5, 0.9$ ）下的任务复杂度度量与单步生成成功率。“Steps”列表示专家演示的平均轨迹长度。“Comp.”为计算得到的复杂度分数（范围 0.1–1.0）。“Diff.”表示定性难度等级。对于每种 α_{disp} 设置，我们将基线方法（RF: ReFlow, SC: ShortCut）与本文方法（MF+Disp）进行比较，“Imp.”显示相对于最佳基线的相对改进。图 9 可视化了这些数据：左图绘制了任务复杂度与最优 α_{disp} 的关系，右图展示了所有任务-权重组合的性能改进热力图。

复杂度定义。 本文通过两个因素的加权组合来定义任务复杂度：（1）轨迹长度复杂度 C_{steps} ，计算为平均步数的最大最小归一化对数；（2）基线难度 C_{rate} ，计算为最大基线成功率倒数的最大最小归一化。最终复杂度分数以相等权重结合这些因素，并重新缩放到 $[0.1, 1.0]$: $C_{\text{final}} = 0.1 + 0.9 \times (0.5 \cdot C_{\text{steps}} + 0.5 \cdot C_{\text{rate}})$ 。

主要发现。 图 13 显示轨迹长度反映了任务复杂度：像 Lift 这样的简单任务表现出围绕较短轨迹的集中分布，而像 Transport 这样的复杂任务则表现出更宽的分布和显著更长的平均长度。表 4 进一步揭示最优分散权重随任务复杂度变化。较简单的任务（Lift、Can）在中等权重（ $\alpha_{\text{disp}} = 0.5$ ）下达到最佳性能，而较复杂的任务（Square、Transport）在较高权重（ $\alpha_{\text{disp}} = 0.9$ ）下受益最大。图 9 用皮尔逊相关系数 $r = 0.924$ 量化了这一关系，证实最优 α_{disp} 随任务复杂度可预测地增加。

表 4. 阶段 1（预训练）：不同分散权重下的任务复杂度与性能改进

Task	Steps	Comp.	Diff.	$\alpha_{\text{disp}} = 0.1$				$\alpha_{\text{disp}} = 0.5$				$\alpha_{\text{disp}} = 0.9$			
				Baseline		Ours		Baseline		Ours		Baseline		Ours	
				RF	SC	MF+Disp	Imp.	RF	SC	MF+Disp	Imp.	RF	SC	MF+Disp	Imp.
Lift	108	0.100	Simplest	78.5%	84.0%	97.0%	19.5%	76.0%	86.5%	98.5%	21.7%	76.0%	86.5%	96.0%	18.7%
Can	224	0.428	Medium	61.0%	62.5%	80.5%	30.4%	73.0%	47.5%	82.5%	43.4%	73.0%	47.5%	77.5%	34.7%
Square	272	0.812	High	30.5%	33.5%	37.5%	17.5%	30.5%	36.5%	37.0%	11.3%	30.5%	36.5%	41.0%	23.4%
Trans.	529	0.978	Highest	39.0%	34.0%	53.0%	45.9%	27.0%	21.0%	34.0%	43.9%	38.0%	21.0%	40.0%	47.9%

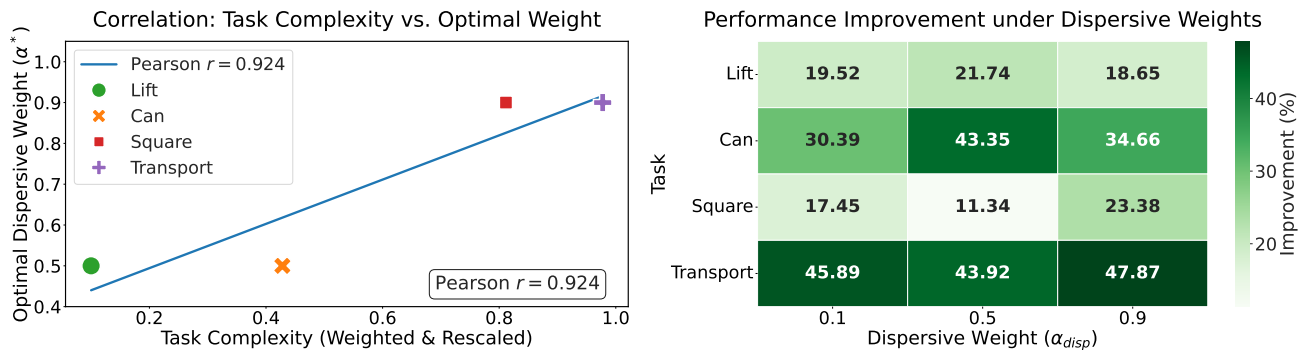


图 9. 阶段 1（预训练）：性能改进分析。左图：任务复杂度与最优分散权重的相关性。右图：跨任务和权重的性能改进热力图。

实用启示。 热图中所有 α_{disp} 取值下的一致性提升表明，分散正则化对超参数不敏感，无论选取何种具体数值，性能均有所改善。作为实用准则，对于更复杂的任务，建议采用更大的 α_{disp} 。这与理论分析一致：复杂任务展现出更多样的动作分布，因此需要更强的表示能力，以避免单步生成过程中的模式坍塌。

A.3. 第二阶段：微调结果

A.3.1. 健身房微调的墙钟时间

墙钟时间比较。为从实用角度评估训练效率，图 10 展示了六项健身房任务的墙钟时间比较。x 轴采用非线性幂变换，以突出差异最为显著的早期训练动态。将采用 1 步推理（NFE=1）和 2 步推理（NFE=2）的 DMPO 与基线方法进行比较，包括 FQL（NFE=1,）、DPPO（NFE=20,）、ReinFlow-R（NFE=4,）和 ReinFlow-S（NFE=4,）。

结果表明，DMPO 在显著减少墙钟时间的同时，实现了具有竞争力或更优的性能。在包括 Hopper、Walker2d、Ant 和 Humanoid 在内的运动任务上，DMPO-1step 比多步基线收敛更快，得益于每次迭代推理成本的降低。值得注意的是，在以成功率衡量的 Kitchen 任务上，与需要每个动作生成 20 个去噪步骤的 DPPO 相比，DMPO 在训练时间大幅缩短的情况下仍保持了强劲性能。

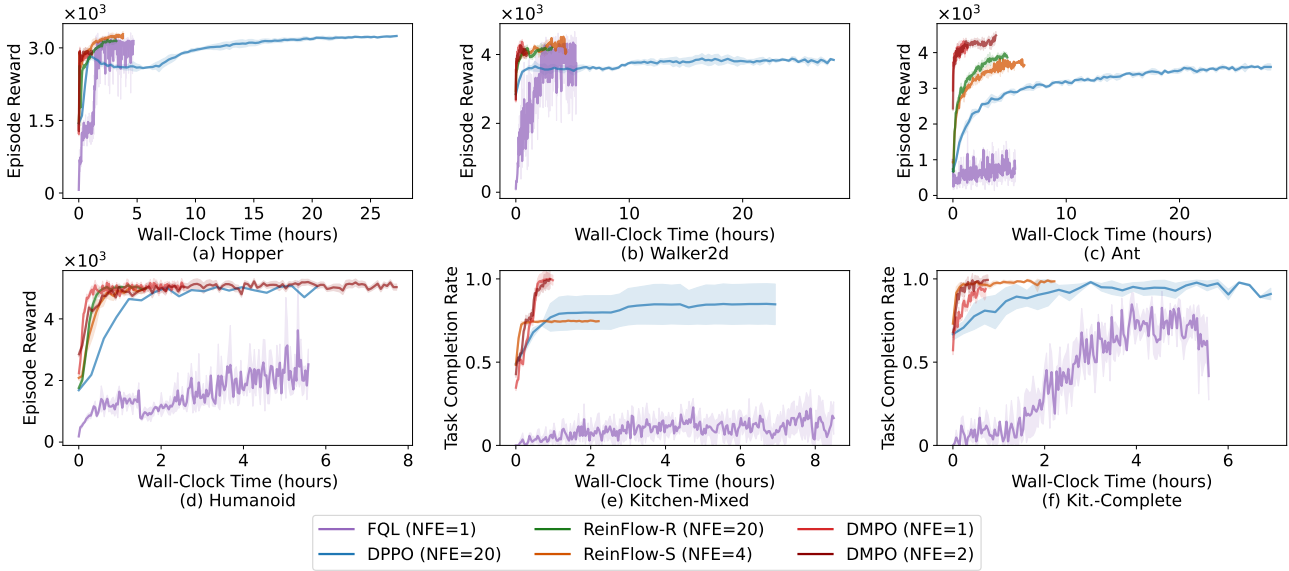


图 10. 第二阶段（PPO 微调）：六项健身房任务的墙钟时间比较。 x 轴表示以小时为单位的训练时间，采用非线性缩放以突出早期训练动态。采用 1 步和 2 步推理的 DMPO（即 NFE=1 和 NFE=2,）实现了具有竞争力的性能，同时与多步基线（如 NFE=20 的 DPPO 和 NFE=4, 的 ReinFlow）相比，所需训练时间大幅减少。效率优势在 Humanoid 和 Kitchen 等复杂任务上尤为显著。

A.3.2. 行为克隆损失消融

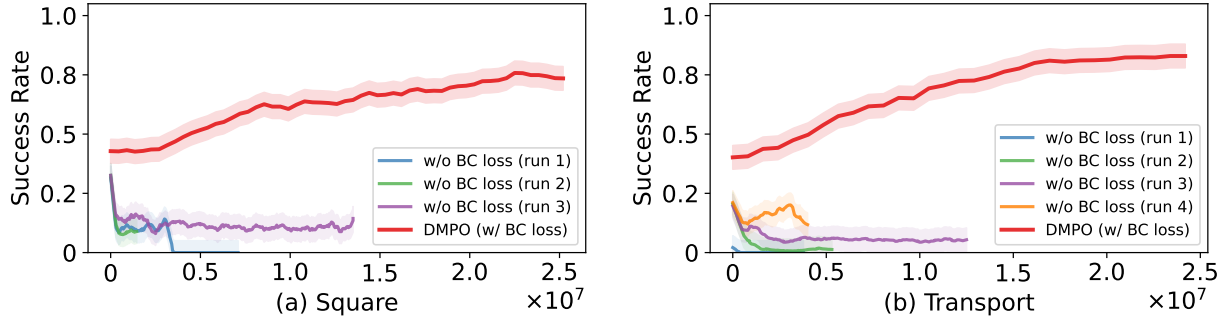


图 11. 行为克隆损失消融实验：在 RoboMimic 的 Square 和 Transport 任务上，比较带行为克隆损失正则化的 DMPO 与不带行为克隆损失的运行结果。不带行为克隆损失时，策略迅速崩溃，而行为克隆正则化使策略能够稳定改进。

我们进一步研究了在 RoboMimic 任务上进行强化学习微调时行为克隆（Behavior Cloning）损失的作用。图 11 比较了

在 Square 和 Transport 任务上，带行为克隆损失正则化的 DMPO 与多次不带行为克隆损失的运行结果。结果清楚地表明，行为克隆损失对于策略的稳定改进至关重要：没有行为克隆正则化时，所有运行的成功率都迅速崩溃至接近零，表明策略偏离预训练分布过远，丧失了已学习的行为。相比之下，带有适当行为克隆损失的 DMPO 能够保持稳定学习，并取得较高的成功率（Square 任务 83%，Transport 任务 87.5%）。这一发现与我们的实用建议一致，即在微调过程中应用适度的、带衰减的行为克隆正则化。

A.4. 物理机器人实验细节

物理机器人实验细节 我们将 DMPO 部署在一台 7 自由度 Franka-Emika-Panda 机器人上，该机器人配备 RealSense D435i 腕部摄像头，分辨率为 $96 \times 96 \times 3$ ，使用 NVIDIA RTX 2080 GPU。我们将 DMPO 与 MP1（MeanFlow Pretrain 1-step，不带色散正则化）进行比较，以评估我们方法的有效性。策略网络以 RGB 图像结合 9 维机器人本体感受状态（包括关节位置和夹爪状态）作为输入，输出 7 维动作，表示末端执行器的增量位姿，具体为 3 维位置和 3 维姿态，以及夹爪指令。

表 5 展示了不同方法在 RTX 2080 上的推理时间分解。每一列分别表示：摄像头（图像采集与预处理）、状态（机器人状态读取）、准备（输入张量准备）、推理（神经网络前向传播）和发送（动作传输至机器人）。值得注意的是，摄像头预处理（包括颜色空间转换和归一化操作）目前在 CPU 上执行，构成了最大的延迟部分（5.4 毫秒）。将这些操作迁移到 GPU 上可以将摄像头处理时间降至 1 毫秒以下，这将进一步将 DMPO 的实时性能提升至总延迟约 5 毫秒。

表 5. 在 RTX 2080 上进行真实机器人部署的推理时间分解（毫秒）。

Method	NFE	Camera	State	Prep.	Inference	Send	Total	Freq (Hz)
Reflow	20				47.7		54.7	18.3
Shortcut	5	5.4	0.1	0.4	12.0	1.1	19.0	52.6
DMPO	5				12.0		19.0	52.6
DMPO	1				2.6		9.6	104.2

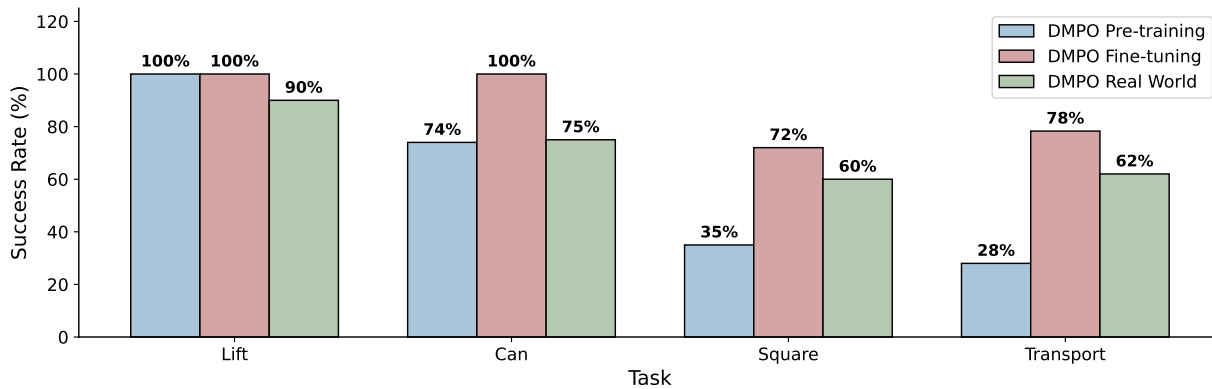


图 12. DMPO 在 RoboMimic 任务上的性能 (NFE=1). 我们在三个阶段评估 DMPO：预训练、微调和真实世界部署。预训练建立基线性能，微调显著提高所有任务的成功率，真实世界实验展示了成功的仿真到现实迁移，并具有竞争力的性能。

图 12 总结了 DMPO 在四个 RoboMimic 任务上三个阶段（预训练、微调和真实世界部署）的成功率。使用行为克隆进行预训练建立了基线性能，在 Lift 上达到 100%，Can 上 74%，Square 上 35%，Transport 上 28%。使用 PPO 进行微调显著提高了所有任务的性能，在 Lift 和 Can 上均达到 100%，Square 上 72%，Transport 上 78%。真实世界部署展示了成功的仿真到现实迁移，成功率分别为 90%（Lift）、75%（Can）、60%（Square）和 62%（Transport）。这些结果验证了 DMPO 的一步策略在物理硬件上部署时保持稳健性能。

B. 数据集分析与可视化

本节对我们实验中使用的数据集进行了全面的分析和可视化，解决了关于数据集选择的潜在问题，并确保与先前工作的公平比较。

B.1. 数据集选择与分布分析

我们的代码库直接构建在 DPPO (Ren 等人, 2025) 和 ReinFlow (Zhang 等人, 2025b) 之上。为了确保与这些流匹配基线的公平比较，我们使用这些工作发布的约 1.9 GB 的简化 RoboMimic 像素数据集，而不是原始扩散策略工作 (Chi 等人, 2023) 中使用的约 78 GB 的全尺寸 RoboMimic 图像数据集。简化数据集是 ReinFlow 在 NeurIPS 2025 和 DPPO 在 ICLR 2025 采用的标准基准，确保我们的性能提升归因于所提出的方法，而不是数据集差异。

RoboMimic 数据集分析。 图 13 对比了三种数据集变体：官方多人类 (MH, 来自多人的 300 条轨迹)、官方熟练人类 (PH, 来自熟练人类的 200 条轨迹) 和简单多人类 (Simple MH, 100 条轨迹)。面板 (a) 展示了图像数据集文件大小，其中搬运 (Transport) 任务数据量最大，官方 MH 达到 31.5GB，而简单 MH 保持紧凑，范围从几十 MB 到 1.4GB。面板 (c) - (f) 呈现轨迹长度分布：提升 (Lift) (c) 是最简单的任务，平均长度为 48 - 108 步，而搬运 (Transport) (f) 最复杂，为 469 - 653 步。MH 数据集因多个演示者而表现出更高的方差，PH 数据集方差较低且轨迹更短，简单 MH 介于两者之间。该分析验证了我们选择简单 MH 的合理性，因为它保留了基本任务特征，同时保持可管理的数据量，展示了任务复杂度层次：提升 < 放置 (Can) < 方形 (Square) < 搬运 (Transport)。

D4RL 数据集分析。 图 14 展示了 D4RL 健身房 (gymnasium) 和厨房 (Kitchen) 任务的分布分析。面板 (a) 显示健身房运动任务每个约含 100 万个样本，而厨房任务样本较少，范围从 3K 到 130K。面板 (b) 揭示健身房任务有 1000 - 2000 条轨迹，而厨房任务仅有 19 - 613 条。面板 (c) 展示回合回报分布，其中蚂蚁-中等-专家 (ant-medium-expert) 获得最高回报且方差较大，厨房任务回报较低但更一致。面板 (d) 说明轨迹长度分布，健身房任务约 1000 步，厨房任务保持 200 - 280 步。这些可视化展示了 D4RL 基准的多样性以及厨房任务因不同子任务序列而产生的多模态特性。

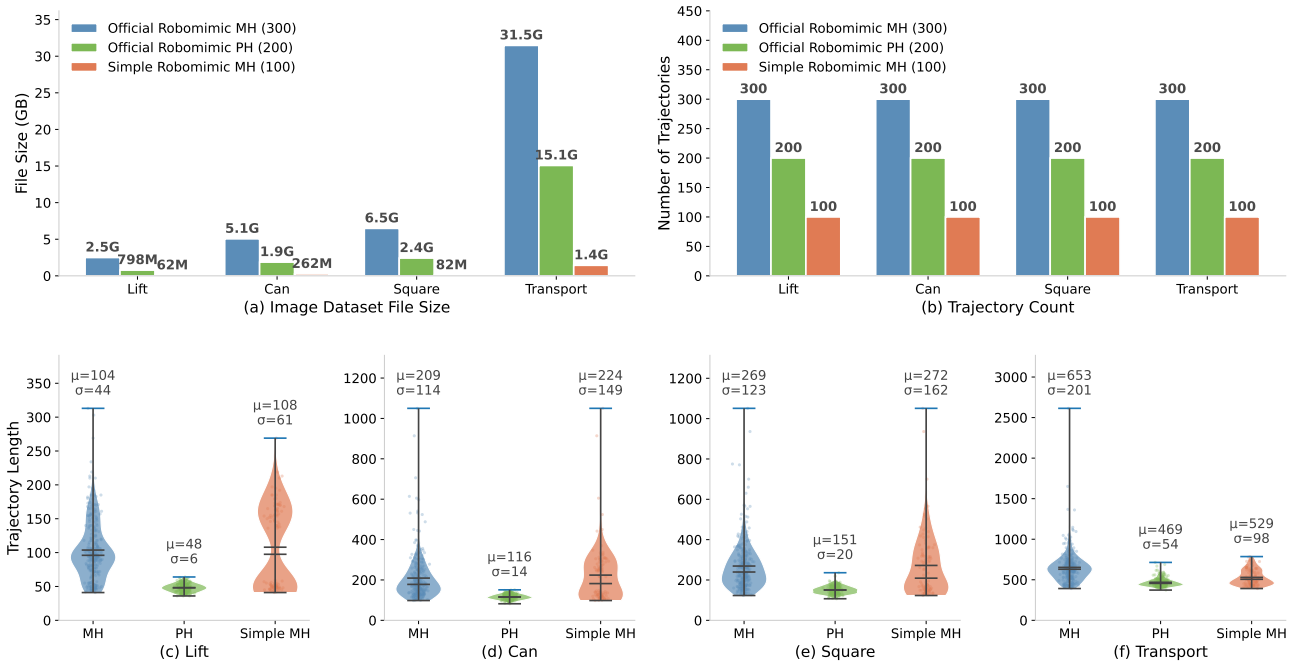


图 13. RoboMimic 数据集分析

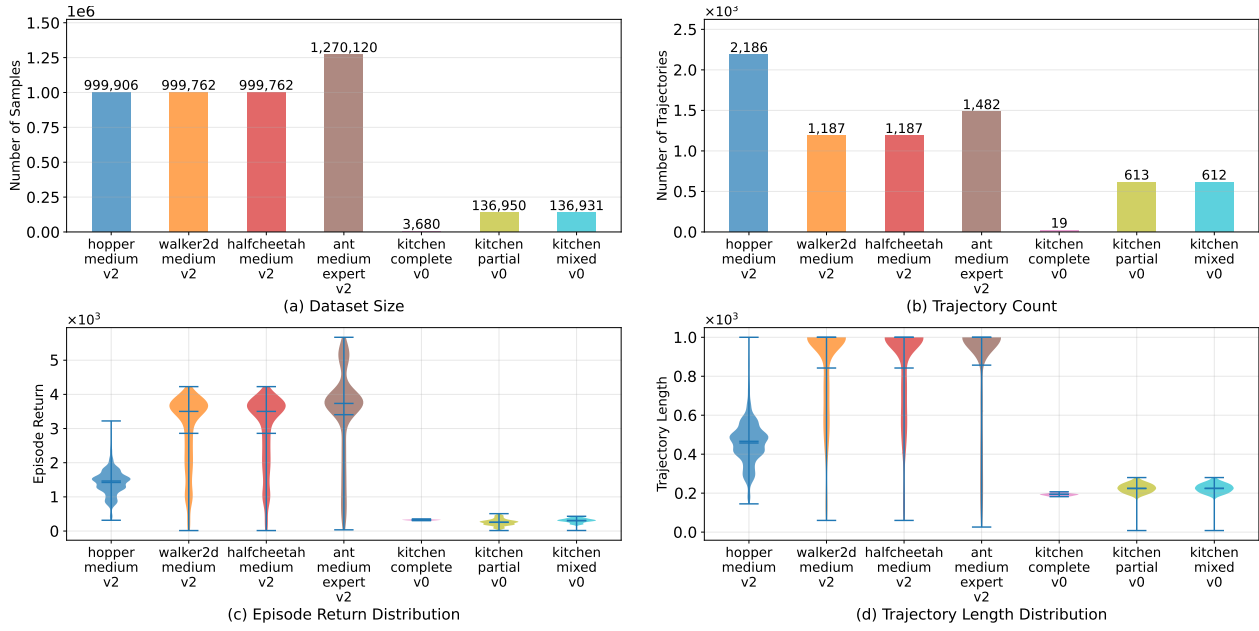


图 14. D4RL 数据集分析

C. 算法与方法公式

本节介绍 DMPO 的核心算法组件：平均流（MeanFlow）策略定义、训练目标、推理过程以及微调损失。

C.1. 阶段 1：平均流预训练

第一阶段利用监督学习在离线演示数据上训练平均流策略。目标是学习一个速度网络 u_θ ，能够将噪声转化为类似专家的动作。

符号适配：源自原始平均流。为了在结合流匹配与强化学习时提高可读性，我们如表 6 所示适配了原始平均流符号。我们的设计优先考虑清晰性如下：

首要优先级：区分最容易混淆的时间相关符号，包括生成模型中的去噪步骤、连续流时间以及强化学习环境时间步。具体来说，我们将原始平均流中的流时间 t 替换为 τ ，以避免与标准强化学习时间步 t 冲突。

次要优先级：对于不太关键的符号（流区间起点 r 和积分变量 ξ ），我们保留原始平均流中的 r ，并使用 ξ 作为积分哑变量（因为 τ 现在已被占用）。在第二阶段中，强化学习奖励通过时间下标 r_t 来区分。

表 6. 符号适配：从原始平均流到避免与强化学习符号冲突。

Concept	Original MeanFlow	This Paper	Reason
Flow time (interval end)	t	τ	Avoid conflict with RL time step t
Flow time (interval start)	r	r	Retained (RL reward uses r_t with subscript)
Integration variable	τ	ξ	τ is now used for flow time endpoint
Interpolated state	z_t	z_τ	Consistent with τ notation

预训练符号说明。表 7 汇总了阶段 1 中使用的符号。

时间约定： $\tau = 0$ 对应目标动作（数据）， $\tau = 1$ 对应纯噪声。这遵循 MeanFlow 约定，采样从 $\tau = 1$ （噪声）进行到 $\tau = 0$ （动作）。

表 7. 阶段 1（预训练）符号汇总。

Concept	Symbol	Description
Flow time (end)	$\tau \in [0, 1]$	Interval endpoint for flow matching
Flow time (start)	$r \in [0, 1]$	Interval start, $r < \tau$
Observation	$o \in \mathcal{O}$	State from environment/dataset
Action	$a \in \mathbb{R}^{d_a}$	Target action from demonstrations
Noise	$\epsilon \sim \mathcal{N}(0, I)$	Standard Gaussian noise
Interpolated state	z_τ	State at flow time τ : $z_\tau = (1 - \tau)a + \tau\epsilon$
Velocity network	u_θ	Neural network predicting average velocity
Network parameters	θ	Learnable parameters
Inference steps	K	Number of denoising iterations

流插值与速度场。

流匹配构建从数据到噪声的连续路径。给定从演示数据集中采样的目标动作 $a \in \mathbb{R}^{d_a}$ 和高斯噪声 $\epsilon \sim \mathcal{N}(0, I)$ ，定义线性插值轨迹：

$$z_\tau = (1 - \tau)a + \tau\epsilon, \quad \tau \in [0, 1] \quad (10)$$

其中 $z_\tau \in \mathbb{R}^{d_a}$ 表示流时间 τ 处的中间状态。在 $\tau = 0$ 处，有 $z_0 = a$ ，对应目标动作。在 $\tau = 1$ 处，有 $z_1 = \epsilon$ ，对应纯噪声。插值系数 $(1 - \tau)$ 控制动作比例，而 τ 控制噪声比例。

瞬时速度场 $v(z_\xi, \xi) : \mathbb{R}^{d_a} \times [0, 1] \rightarrow \mathbb{R}^{d_a}$ 描述了每个点沿流动轨迹的变化率。区间 $[r, \tau]$ 上的平均速度场，其中 $0 \leq r < \tau \leq 1$ ，定义为：

$$u(z_\tau, r, \tau) = \frac{1}{\tau - r} \int_r^\tau v(z_\xi, \xi) d\xi \quad (11)$$

该平均速度表示时间区间 $[r, \tau]$ 内流动的平均方向和大小。重要的是，它满足位移恒等式：

$$(\tau - r) u(z_\tau, r, \tau) = z_\tau - z_r \quad (12)$$

该恒等式表明，在区间上对平均速度积分恰好恢复 z_r 与 z_τ 之间的位移。这一恒等式对于平均流进行任意步长推理的能力至关重要：通过预测平均速度 u ，可以直接计算状态转移，而无需对瞬时速度 v 进行数值积分。

平均流恒等式的推导。现在从位移恒等式推导平均流恒等式。定义累积位移函数：

$$F(\tau) = \int_r^\tau v(z_\xi, \xi) d\xi = z_\tau - z_r \quad (13)$$

根据公式 11 中平均速度的定义，有：

$$u(z_\tau, r, \tau) = \frac{F(\tau)}{\tau - r} \quad (14)$$

使用商法则对 τ 求导：

$$\frac{d}{d\tau} u = \frac{F'(\tau)(\tau - r) - F(\tau) \cdot 1}{(\tau - r)^2} \quad (15)$$

根据微积分基本定理， $F'(\tau) = v(z_\tau, \tau)$ 。代入并化简：

$$\begin{aligned} \frac{d}{d\tau} u &= \frac{v(z_\tau, \tau)(\tau - r) - F(\tau)}{(\tau - r)^2} = \frac{v(z_\tau, \tau)}{\tau - r} - \frac{F(\tau)}{(\tau - r)^2} \\ &= \frac{v(z_\tau, \tau)}{\tau - r} - \frac{u}{\tau - r} = \frac{v(z_\tau, \tau) - u}{\tau - r} \end{aligned} \quad (16)$$

重排 $(\tau - r) \frac{d}{d\tau} u = v(z_\tau, \tau) - u$ yields the **MeanFlow identity**:

$$u(z_\tau, r, \tau) = v(z_\tau, \tau) - (\tau - r) \frac{d}{d\tau} u(z_\tau, r, \tau) \quad (17)$$

该恒等式表明，平均速度等于瞬时速度减去一个修正项 $(\tau - r) \frac{d}{d\tau} u$ ，该修正项描述了平均速度随时间的变化。当平均速度恒定时，修正项消失， $u = v$ 。

平均流训练目标。平均流损失训练网络预测将噪声传输到目标动作的平均速度：

$$\mathcal{L}_{\text{MF}}(\theta) = \mathbb{E}_{(o,a) \sim \mathcal{D}, \epsilon \sim \mathcal{N}(0,I), (r,\tau)} [\|u_\theta(z_\tau, r, \tau, o) - \text{sg}(u_{\text{tgt}})\|^2] \quad (18)$$

其中 $\text{sg}(\cdot)$ 表示停止梯度算子，在反向传播过程中将其参数视为常数 (Chen & He, 2021), $\mathcal{D} = \{(o_i, a_i)\}_{i=1}^N$ 是离线演示数据集， (r, τ) 是随机采样的时间区间端点，满足 $0 \leq r < \tau \leq 1$ ， $z_\tau = (1 - \tau)a + \tau\epsilon$ 是插值状态 (式 10)。网络 $u_\theta(z_\tau, r, \tau, o)$ 预测区间 $[r, \tau]$ 上的平均速度， u_{tgt} 是由平均流恒等式导出的目标速度。

计算目标速度。将平均流恒等式 (式 17) 应用于训练需要计算全导数。对于线性插值，瞬时速度是恒定的： $\frac{d}{d\tau} u_\theta \big|_{v_\tau = \epsilon - a}$ 。由于 $u_\theta(z_\tau, r, \tau, o)$ 既显式地又通过 $z_\tau = (1 - \tau)a + \tau\epsilon$ 隐式地依赖于 τ ，本文应用多元链式法则：

$$\frac{d}{d\tau} u_\theta(z_\tau, r, \tau, o) = \frac{dz_\tau}{d\tau} \cdot \partial_z u_\theta + \frac{dr}{d\tau} \cdot \partial_r u_\theta + \frac{d\tau}{d\tau} \cdot \partial_\tau u_\theta \quad (19)$$

Substituting $\frac{dz_\tau}{d\tau} = v_\tau = \epsilon - a$, $\frac{dr}{d\tau} = 0$ (因为 r 在采样后固定)，且 $\frac{d\tau}{d\tau} = 1$:

$$\frac{d}{d\tau} u_\theta(z_\tau, r, \tau, o) = v_\tau \cdot \nabla_z u_\theta + \partial_\tau u_\theta \quad (20)$$

这是雅可比矩阵 $[\partial_z u_\theta, \partial_r u_\theta, \partial_\tau u_\theta]$ 与输入 (z_τ, r, τ) 的切向量 $(v_\tau, 0, 1)$ 之间的雅可比-向量积 (JVP)。代入式 17，目标速度为：

$$u_{\text{tgt}} = v_\tau - (\tau - r) \frac{d}{d\tau} u_\theta = (\epsilon - a) - (\tau - r) (v_\tau \cdot \nabla_z u_\theta + \partial_\tau u_\theta) \quad (21)$$

注意 u_{tgt} 依赖于当前网络 u_θ ，这使得该训练目标具有自洽性。式 18 中的停止梯度算子 $\text{sg}(\cdot)$ 至关重要：它在反向传播过程中将 u_{tgt} 视为固定目标，防止梯度流经 JVP 项时不稳定。这类似于 DQN 中的目标网络或 SimSiam (Chen & He, 2021) 中的停止梯度机制。训练完成后，网络支持一步生成：给定噪声 $z_1 \sim \mathcal{N}(0, I)$ ，直接计算动作为 $a = z_1 - u_\theta(z_1, r=0, \tau=1, o)$ 。

对于时间采样，我们使用对数正态分布，其中 $\tau, r = \text{sigmoid}(\xi_1), \text{sigmoid}(\xi_2)$ 进行排序， $\xi_1, \xi_2 \sim \mathcal{N}(0, 1)$ ，将样本集中在 $[0, 1]$ 的中间区域，避免边界效应。一小部分 ($\rho = 0.1$) 样本设置 $r = \tau$ ，为瞬时速度预测提供监督。

C.2. 阶段 1：带分散正则化的平均流预训练

我们引入分散正则化以防止表示坍塌，即编码器将不同观测映射到几乎相同表示的现象 (Wang & He, 2025)。分散正则化对于一步生成模型尤为重要，因为网络必须直接将噪声映射到多样化的动作，而无需迭代细化。其核心思想类似于对比自监督学习，鼓励内部表示在隐藏空间中分散，但不需要正样本对。

即插即用特性：分散正则化仅用于训练阶段。它在预训练期间向损失函数添加正则化项，但在推理阶段无需任何修改。训练完成后，正则化项被丢弃，模型执行标准平均流采样。这使得它易于集成到现有流程中，不影响推理速度或架构。

符号说明。 设 B 表示批大小， $\{o_i\}_{i=1}^B$ 表示一批观测值。我们从速度网络 u_θ 的选定层中提取中间表示。遵循 Wang & He (2025)，设 $h_i = f_\theta(o_i) \in \mathbb{R}^{d_h}$ 表示观测 o_i 的展平中间激活，其中 f_θ 表示直到选定中间块的层子集。每个观测产生一个表示向量，我们将它们收集到批表示矩阵中。温度超参数 $\tau > 0$ （通常为 $\tau = 0.1$ ）控制分散损失的锐度 $\mathbf{H} = [h_1, \dots, h_B]^\top \in \mathbb{R}^{B \times d_h}$ 度。

我们提供四种分散正则化变体：

InfoNCE-L2 损失。 鼓励表示在欧氏距离上彼此远离：

$$\mathcal{L}_{\text{NCE-L2}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\|h_i\|_2^2/\tau)}{\sum_{j \neq i} \exp(-\|h_i - h_j\|_2^2/\tau)} \quad (22)$$

分子鼓励较大的表示范数，而分母惩罚较小的成对距离 $\|h_i - h_j\|_2^2$ 。最小化此损失会将表示推开。

InfoNCE-余弦损失。 鼓励表征具有多样化的方向（角度分离）：

$$\mathcal{L}_{\text{NCE-Cos}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(1/\tau)}{\sum_{j \neq i} \exp(\text{sim}_{\cos}(h_i, h_j)/\tau)} \quad (23)$$

其中两个向量之间的余弦相似度为：

$$\text{sim}_{\cos}(h_i, h_j) = \frac{h_i^\top h_j}{\|h_i\|_2 \cdot \|h_j\|_2} \in [-1, 1]$$

常数分子 $\exp(1/\tau)$ 作为参考。最小化该损失会降低所有向量对之间的余弦相似度，使表征在超球面上均匀分布。

合页损失。 直接惩罚距离小于边界 m 的向量对：

$$\mathcal{L}_{\text{Hinge}} = \frac{1}{B(B-1)} \sum_{i=1}^B \sum_{j \neq i} \max(0, m - \|h_i - h_j\|_2) \quad (24)$$

其中 $m > 0$ 是边界超参数，通常为 $m = 1.0$ ，表示任意两个表征之间的最小期望距离， $\max(0, \cdot)$ 是合页函数，仅惩罚距离小于 m 的向量对， $B(B-1)$ 是用于归一化的有序对数量。

协方差正则化。 鼓励表征的协方差矩阵为对角矩阵，特征之间去相关：

$$\mathcal{L}_{\text{Cov}} = \frac{1}{d_h} \sum_{i=1}^{d_h} \sum_{j \neq i} [C(\mathbf{H})]_{ij}^2 \quad (25)$$

其中样本协方差矩阵为：

$$C(\mathbf{H}) = \frac{1}{B-1} (\mathbf{H} - \bar{\mathbf{H}})^\top (\mathbf{H} - \bar{\mathbf{H}}) \in \mathbb{R}^{d_h \times d_h}$$

其中 $\bar{\mathbf{H}} = \frac{1}{B} \sum_{i=1}^B h_i$ 为广播到所有行的批次均值。该损失惩罚非对角元素 $[C(\mathbf{H})]_{ij}$ （对于 $i \neq j$ ），鼓励特征维度之间不相关。

第一阶段组合目标。 第一阶段训练总损失结合了平均流重构与分散正则化：

$$\mathcal{L}_{\text{Stage1}} = \mathcal{L}_{\text{MF}} + \alpha_{\text{disp}} \mathcal{L}_{\text{disp}}(\mathbf{H}^{(\text{Cond})}) \quad (26)$$

其中 $\mathcal{L}_{\text{MF}}$ 是式中的平均流速度预测损失 [18], $\mathcal{L}_{\text{disp}} \in \{\mathcal{L}_{\text{NCE-L2}}, \mathcal{L}_{\text{NCE-Cos}}, \mathcal{L}_{\text{Hinge}}, \mathcal{L}_{\text{Cov}}\}$ 是应用于条件嵌入的选定色散正则化 $\mathbf{H}^{(\text{Cond})}$ 由视觉变换器编码器输出, 且 $\alpha_{\text{disp}} > 0$ 是平衡重建准确率与表示多样性的权重, 典型值为 $\alpha_{\text{disp}} = 0.1$. DMPO 默认使用 InfoNCE-L2 损失, 因为它在原始色散正则化研究 (Wang & He, 2025) 以及我们机器人操作任务实验中都达到了最佳性能。

推理过程。 平均流的一个关键优势是, 同一个训练好的网络 u_θ 支持任意推理步数 $K \in \{1, 5, 32, 128, \dots\}$, 无需重新训练。这种灵活性源于 $u_\theta(z_\tau, r, \tau, o)$ 被训练为预测任意区间 $[r, \tau]$ 上的平均速度, 其中 τ 表示区间终点, r 表示区间起点, 而不是固定的离散化。在部署时, 我们可以根据可用的计算预算和所需的动作质量来选择 K 。

给定任意步数预算 $K \geq 1$, 我们将单位时间区间 $[0, 1]$ 划分为 K 个相等的段。定义时间调度:

$$1 = \tau_0 > \tau_1 > \tau_2 > \dots > \tau_K = 0, \quad \text{其中 } \tau_k = 1 - \frac{k}{K}$$

多步推理从噪声到动作迭代地细化状态:

$$z_{\tau_{k+1}} = z_{\tau_k} - (\tau_k - \tau_{k+1}) u_\theta(z_{\tau_k}, \tau_{k+1}, \tau_k, o), \quad k = 0, \dots, K-1 \quad (27)$$

其中 $z_{\tau_0} = z_1 \sim \mathcal{N}(0, I)$ 是来自标准高斯的初始样本, 表示在 $\tau = 1, z_{\tau_k} \in \mathbb{R}^{d_a}$ 是步骤 $k, (\tau_k - \tau_{k+1}) = 1/K$ 是时间步长, $u_\theta(z_{\tau_k}, \tau_{k+1}, \tau_k, o)$ 是区间 $[\tau_{k+1}, \tau_k]$ 的预测平均速度, 第一个时间参数为区间起点, 第二个为区间终点, 且 $a = z_{\tau_K} = z_0$ 是经过 K 个去噪步骤后的最终动作。

关于时间方向的说明: 在流匹配中, 我们在训练期间构建从 $\tau = 0$ 处的数据到 $\tau = 1$ 处的噪声的轨迹 (i.e., $z_\tau = (1 - \tau)a + \tau\epsilon$)。在推理期间, 我们反转此过程: 从对应于噪声的 $\tau = 1$ 开始, 向对应于目标动作的 $\tau = 0$ 积分。更新从当前状态中减去 $(\tau_k - \tau_{k+1}) \cdot u_\theta$ 以向目标移动。

一步推理 ($K = 1$). 特殊情况 $K = 1$ 将整个去噪过程折叠为单次前向传递:

$$a_{\text{pred}} = z_1 - u_\theta(z_1, r=0, \tau=1, o), \quad z_1 \sim \mathcal{N}(0, I) \quad (28)$$

其中 $z_1 \sim \mathcal{N}(0, I)$ 是作为起点的采样噪声, $u_\theta(z_1, r=0, \tau=1, o)$ 是整个区间 $[0, 1]$ 上的预测平均速度, 以 $\tau = 1$ 为区间终点、 $r = 0$ 为区间起点, $a_{\text{pred}} \in \mathbb{R}^{d_a}$ 是一步预测的动作。

该公式完全消除了迭代常微分方程积分, 使得能够以 90Hz 或更高的频率进行实时控制。网络直接输出从噪声到动作的完整位移, 使推理速度与标准前馈网络一样快。

权衡: 单步推理速度最快, 但与多步推理相比, 对于复杂的动作分布, 可能会牺牲一定的动作质量。C.1 节中描述的分散正则化对于在单步推理下保持质量尤为重要。

C.3. 第二阶段：近端策略优化微调

强化学习微调的符号表示。 为避免环境时间步与流时间之间的混淆, 本文采用以下符号约定: t 表示离散环境时间步, 而 τ 表示连续流时间。这一区分与 ReinFlow (Zhang 等人, 2025b) 保持一致。符号消歧: 字母 r 用于两个不同的概念: (1) 第一阶段及去噪链中的流区间起点 r 或 r_k , 以及 (2) 环境奖励 r_t 。二者通过上下文和下标加以区分: 不带下标或带去噪索引 k 的 r 指流时间, 而带环境时间下标 t 的 r_t 指奖励。表 8 总结了第二阶段使用的符号。

时间约定 (与第一阶段一致): 流时间约定与第一阶段相同: $\tau = 0$ 对应目标动作 (数据), $\tau = 1$ 对应纯噪声。在离散设置中, $\tau_0 = 1$ (噪声) 和 $\tau_K = 0$ (动作)。因此, 去噪链从 $\tau_0 = 1$ 处的 $a^0 \sim \mathcal{N}(0, I)$ 开始, 到 $\tau_K = 0$ 处的最终动作 a^K 结束。

与第一阶段符号的关系: 在第二阶段 (预训练) 中, 我们使用连续流时间 $\tau \in [0, 1]$ 和区间起点 r , 并采用插值状态 $z_\tau = (1 - \tau)a + \tau\epsilon$ 。在第二阶段, 流时间被离散化为 $\tau_k = 1 - k/K$, 用于去噪步骤 k , 其中

表 8. 第二阶段（强化学习微调）符号汇总。

Concept	Symbol	Description
Environment time step	t	Step index within an episode
Denoising step	k (superscript)	Step index in the denoising chain
Flow time (end)	$\tau_k = 1 - k/K$	Interval endpoint at step k
Flow time (start)	$r_k = \tau_{k+1}$	Interval start at step k
Training iteration	n	PPO iteration index
Observation	o_t	Observation at environment step t
Final action	a_t	Action executed at step t
Intermediate action	a_t^k	Action at denoising step k
Reward	r_t	Reward at environment step t
PPO probability ratio	$\rho_t(\theta)$	Ratio of new to old policy

中间动作 a^k 对应于 z_{τ_k} 。第一阶段的区间 $[r, \tau]$ 对应于第二阶段的 $[\tau_{k+1}, \tau_k]$ 。在每个环境步骤 t 内，去噪链在固定观测 $o = o_t$ 下运行。

第二阶段利用在线强化学习对预训练的 MeanFlow 策略进行微调。本文采用近端策略优化（Proximal Policy Optimization, PPO），通过截断代理目标实现稳定的策略更新。为支持 PPO 训练，本文首先将多步去噪过程形式化为具有可处理对数概率的马尔可夫链。

多步马尔可夫策略。 MeanFlow 将动作生成形式化为一个 K 步马尔可夫链，从而为策略梯度方法提供可处理的概率计算。在单个环境步骤 t 内，观测 $o = o_t$ 在整个去噪链期间保持不变。为简化符号，以下推导中省略下标 t 。

定义 C.1（多步马尔可夫策略）。给定环境步骤 t ，MeanFlow 通过 o (i.e., o_t 处的观测) 生成 K 步去噪过程生成动作：

$$a^0 \sim p_0(\cdot) = \mathcal{N}(0, I), \quad a^{k+1} \sim \pi_\theta(a^{k+1} | a^k, o), \quad k = 0, \dots, K-1 \quad (29)$$

其中 $a^0 \in \mathbb{R}^{d_a}$ 是来自先验分布 $p_0 = \mathcal{N}(0, I)$, $a^k \in \mathbb{R}^{d_a}$ 的初始样本，表示步骤 k 处的中间动作，该动作从噪声逐步细化到最终动作， $\pi_\theta(a^{k+1} | a^k, o)$ 是由 θ 参数化的学习转移核， $K \in \mathbb{N}^+$ 是去噪步骤的总数，例如 1、5、32 等。在环境中执行的最终动作是 $a := a^K$ 。

马尔可夫链中的每个转移都遵循以确定性更新为中心的高斯分布：

$$a^{k+1} \sim \mathcal{N}(a^k - \Delta \tau_k \cdot u_\theta(a^k, \tau_{k+1}, \tau_k, o), \sigma^2 I) \quad (30)$$

其中 $\tau_k = 1 - k/K$ 是步骤 k 处的流时间，从 $\tau_0 = 1$ 递减到 $\tau_K = 0$ ， $\Delta \tau_k = \tau_k - \tau_{k+1} = 1/K$ 是均匀划分下的时间步长， $u_\theta(a^k, \tau_{k+1}, \tau_k, o)$ 是神经网络，在给定当前状态 a^k 和观测 o 的条件下预测区间 $[\tau_{k+1}, \tau_k]$ 上的平均速度（第一个时间参数 τ_{k+1} 是区间起点，第二个 τ_k 是区间终点）， $\sigma^2 > 0$ 是转移噪声的方差，这是一个小的超参数，通常设置为 $\sigma = 0.01$ ，以确保策略具有完全支持以进行探索。

整个轨迹 $a^{0:K} = (a^0, a^1, \dots, a^K)$ 的联合对数概率分解为：

$$\ln \pi_\theta(a^{0:K} | o) = \ln p_0(a^0) + \sum_{k=0}^{K-1} \ln \mathcal{N}(a^{k+1} | \mu_k, \sigma^2 I) \quad (31)$$

其中 $\mu_k = a^k - \Delta \tau_k \cdot u_\theta(a^k, \tau_{k+1}, \tau_k, o)$ 是预测均值。每个高斯对数概率项具有闭式形式：

$$\ln \mathcal{N}(x | \mu, \sigma^2 I) = -\frac{d_a}{2} \ln(2\pi\sigma^2) - \frac{\|x - \mu\|_2^2}{2\sigma^2}$$

这种可处理的对数概率使得可以直接应用策略梯度算法，如 PPO。

标准强化学习符号： 设 $\gamma \in [0, 1)$ 表示未来奖励的折扣因子，典型值为 $\gamma = 0.99$ ， $\sum_{t=0}^{\infty} \gamma^t r_t$ 表示期望的累积折扣回报。状态价值函数 $V^\pi(o) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_t \mid o_0 = o]$ 估计从状态 o 开始的期望回报，而状态-动作价值函数 $Q^\pi(o, a) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_t \mid o_0 = o, a_0 = a]$ 估计在状态 o 中采取动作 a 后的期望回报。优势函数 $A^\pi(o, a) = Q^\pi(o, a) - V^\pi(o)$ 衡量动作 a 比平均水平好多少。

多步策略的策略梯度。 针对平均流的多步动作生成，本文推导出一种策略梯度，将最终动作的优势归功于所有中间去噪步骤。

定理 C.2（多步策略梯度）。设 π_θ 为平均流策略，通过 K 步马尔可夫链生成动作，如公式 29 所定义。策略梯度为：

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{o, a^{0:K} \sim \pi_\theta} \left[A^{\pi_\theta}(o, a^K) \nabla_\theta \sum_{k=0}^{K-1} \ln \pi_\theta(a^{k+1} | a^k, o) \right] \quad (32)$$

其中 o 为策略执行过程中采样的观测， $a^{0:K} = (a^0, a^1, \dots, a^K) \sim \pi_\theta(\cdot | o)$ 为从策略中采样的完整去噪轨迹， $A^{\pi_\theta}(o, a^K)$ 为最终动作 a^K 的优势，该动作是环境中实际执行的动作， $\sum_{k=0}^{K-1} \ln \pi_\theta(a^{k+1} | a^k, o)$ 为所有 K 次转移的对数概率之和。

关键洞见： 由环境奖励计算得到的最终动作优势 $A(o, a^K)$ 被用于更新所有中间转移概率。这是有效的，因为所有转移都对最终动作的产生做出了贡献：若 a^K 产生高奖励，则应强化产生该动作的整个去噪链。

近端策略优化（PPO）裁剪替代目标。 PPO 通过限制动作概率的变化来稳定策略更新。定义概率比：

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)} = \exp(\log \pi_\theta(a_t | o_t) - \log \pi_{\theta_{\text{old}}}(a_t | o_t)) \quad (33)$$

其中 π_θ 是当前正在优化的策略， $\pi_{\theta_{\text{old}}}$ 是用于收集训练数据的策略（在优化期间冻结）， a_t, o_t 是收集轨迹中环境步骤 t 处的动作和观测。

裁剪代理策略损失可防止过大的策略更新：

$$\mathcal{L}_{\text{PG}}(\theta) = \mathbb{E}_t [\max(-A_t \cdot \rho_t(\theta), -A_t \cdot \text{clip}(\rho_t(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}))] \quad (34)$$

其中 A_t 是环境步骤 t 处使用如下所述 GAE 计算的估计优势， $\epsilon_{\text{clip}} > 0$ 是裁剪阈值，典型值为 $\epsilon_{\text{clip}} = 0.2$ ， $\text{clip}(x, a, b) = \min(\max(x, a), b)$ 将 x 裁剪到区间 $[a, b]$ 。对未裁剪和裁剪目标取最大值可确保策略仅在改善目标时才更新，并且与 $\pi_{\theta_{\text{old}}}$ 的较大偏差会受到惩罚。

广义优势估计（GAE）： 我们使用参数为 $\lambda_{\text{GAE}} \in [0, 1]$ 的 GAE 估计优势，通常 $\lambda_{\text{GAE}} = 0.95$ ：

$$\hat{A}_t = \sum_{\ell=0}^{\infty} (\gamma \lambda_{\text{GAE}})^\ell \delta_{t+\ell}, \quad \text{其中 } \delta_t = r_t + \gamma V_\theta(o_{t+1}) - V_\theta(o_t)$$

是 TD 残差。GAE 提供了由 λ_{GAE} 控制的偏差-方差权衡。实际上，对于在步骤 T 结束的有限回合，这变为：

$$\hat{A}_t = \sum_{\ell=0}^{T-t-1} (\gamma \lambda_{\text{GAE}})^\ell \delta_{t+\ell}$$

可以通过反向递归高效计算： $\hat{A}_{T-1} = \delta_{T-1}$ 和 $\hat{A}_t = \delta_t + \gamma \lambda_{\text{GAE}} \hat{A}_{t+1}$ 对于 $t < T-1$ 。

价值函数损失。 价值网络 $V_\theta : \mathcal{O} \rightarrow \mathbb{R}$ 估计期望回报。我们训练它以最小化价值损失：

$$\mathcal{L}_V(\theta) = \frac{1}{2} \mathbb{E}_t [(V_\theta(o_t) - R_t)^2] \quad (35)$$

其中 $V_\theta(o_t) \in \mathbb{R}$ 是对观测 o_t , $R_t = \sum_{\ell=0}^{T-t}$ 的预测价值, $\gamma^\ell r_{t+\ell}$ 是经验回报, 表示从环境步 t 到回合结束 T 的折扣奖励之和, 因子 $\frac{1}{2}$ 将梯度简化为 $(V_\theta(o_t) - R_t)$ 。

熵正则化。为鼓励探索并防止过早收敛, 我们添加熵奖励:

$$\mathcal{L}_{\text{ent}}(\theta) = -\mathbb{E}_t[H(\pi_\theta(\cdot | o_t))] \quad (36)$$

其中策略分布的熵为:

$$H(\pi_\theta(\cdot | o)) = -\mathbb{E}_{a \sim \pi_\theta(\cdot | o)}[\log \pi_\theta(a | o)]$$

对于 MeanFlow, 由于每个转移如式 30 所定义为高斯分布, 熵具有闭式形式:

$$H(\mathcal{N}(\mu, \sigma^2 I)) = \frac{d_a}{2} (1 + \log(2\pi\sigma^2))$$

最小化 $\mathcal{L}_{\text{ent}}$ (其带有负号) 可最大化熵, 从而鼓励多样化的动作探索。

行为克隆正则化。为防止微调后的策略偏离预训练的专家行为过远, 本文引入行为克隆 (Behavior Cloning) 或 BC 损失:

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{o \sim \mathcal{D}_{\text{rollout}}} [\|a_\omega(o) - a_\theta(o)\|_2^2] \quad (37)$$

其中 $a_\omega(o) \in \mathbb{R}^{d_a}$ 是来自冻结的预训练策略的动作, 其参数 ω 在微调期间保持固定, $a_\theta(o) \in \mathbb{R}^{d_a}$ 是来自当前微调策略的动作, 其参数为 θ , $\mathcal{D}_{\text{rollout}}$ 表示在线推演过程中收集的观测。两个策略使用相同的噪声种子 z_1 进行评估, 以确保比较有意义。

BC 系数遵循线性衰减计划, 随着学习进行逐步降低正则化强度:

$$\lambda_{\text{BC}}(n) = \begin{cases} \lambda_{\text{BC}}^{\text{init}}, & n < n_{\text{start}} \\ \lambda_{\text{BC}}^{\text{init}} + \frac{n - n_{\text{start}}}{n_{\text{end}} - n_{\text{start}}} (\lambda_{\text{BC}}^{\text{final}} - \lambda_{\text{BC}}^{\text{init}}), & n_{\text{start}} \leq n < n_{\text{end}} \\ \lambda_{\text{BC}}^{\text{final}}, & n \geq n_{\text{end}} \end{cases} \quad (38)$$

其中 n 是当前训练迭代次数, $\lambda_{\text{BC}}^{\text{init}}$ 是初始 BC 权重, 通常设为 1.0 以在早期提供强正则化, $\lambda_{\text{BC}}^{\text{final}}$ 是最终 BC 权重, 通常设为 0.0 以允许完全的强化学习优化, $n_{\text{start}}, n_{\text{end}}$ 定义了衰减发生的迭代范围。

第二阶段联合目标。第二阶段微调的总损失结合了所有组成部分:

$$\mathcal{L}_{\text{Stage2}}(\theta) = \mathcal{L}_{\text{PG}}(\theta) + \lambda_V \cdot \mathcal{L}_V(\theta) + \lambda_{\text{ent}} \cdot \mathcal{L}_{\text{ent}}(\theta) + \lambda_{\text{BC}}(n) \cdot \mathcal{L}_{\text{BC}}(\theta) \quad (39)$$

其中 $\mathcal{L}_{\text{PG}}$ 是式中的近端策略优化裁剪策略梯度损失 34, $\mathcal{L}_V$ 是式中的价值函数均方误差损失 35, $\mathcal{L}_{\text{ent}}$ 是式 36 中用于探索的负熵, 且 $\mathcal{L}_{\text{BC}}$ 是式 37 中的行为克隆正则化。系数为 $\lambda_V > 0$ 用于价值损失, 典型值为 $\lambda_V = 0.5$, $\lambda_{\text{ent}} > 0$ 用于熵, 典型值为 $\lambda_{\text{ent}} = 0.01$, 且 $\lambda_{\text{BC}}(n)$ 针对式 (38) 中随迭代变化的边界条件系数。策略网络与价值网络通过最小化联合更新 $\mathcal{L}_{\text{Stage2}}$ 采用梯度下降法, 通常使用 Adam 优化器。

算法伪代码。

Algorithm 1 DMPO Stage 1: Pre-Training with Dispersive Regularization

Require: Dataset $\mathcal{D} = \{(o, a)\}$, dispersive weight α_{disp}
Ensure: Trained policy network π_θ

- 1: Initialize network parameters θ
- 2: **for** each epoch **do**
- 3: **for** each batch $\mathcal{B} = \{(o^{(i)}, a^{(i)})\}_{i=1}^B$ **do**
- 4: Sample $\epsilon^{(i)} \sim \mathcal{N}(0, I)$ and $(r^{(i)}, \tau^{(i)})$ from logit-normal
- 5: Compute interpolants: $z_\tau^{(i)} = (1 - \tau^{(i)})a^{(i)} + \tau^{(i)}\epsilon^{(i)}$
- 6: Predict velocity: $\hat{u}^{(i)} = u_\theta(z_\tau^{(i)}, r^{(i)}, \tau^{(i)}, o^{(i)})$
- 7: Compute target: $u_{\text{tgt}}^{(i)} = (\epsilon^{(i)} - a^{(i)}) - (\tau^{(i)} - r^{(i)}) \frac{d}{d\tau} u_\theta$ {via JVP, Eq. 21}
- 8: Extract representations: $\mathbf{h}^{(i)} = \phi_\theta(o^{(i)})$
- 9: $\mathcal{L}_{\text{MF}} = \frac{1}{B} \sum_i \|\hat{u}^{(i)} - \text{sg}(u_{\text{tgt}}^{(i)})\|_2^2$ {sg: stop-gradient}
- 10: $\mathcal{L}_{\text{disp}} = \text{DispersiveLoss}(\{\mathbf{h}^{(i)}\})$
- 11: $\theta \leftarrow \theta - \eta \nabla_\theta (\mathcal{L}_{\text{MF}} + \alpha_{\text{disp}} \mathcal{L}_{\text{disp}})$
- 12: **end for**
- 13: **end for**

Algorithm 2 DMPO Inference (One-Step or Multi-Step)

Require: Observation o , trained policy π_θ , number of steps K
Ensure: Action a

- 1: Sample $z_1 \sim \mathcal{N}(0, I)$
- 2: **if** $K = 1$ **then**
- 3: $a = z_1 - u_\theta(z_1, 0, 1, o)$ {One-step: $r=0, \tau=1$ }
- 4: **else**
- 5: **for** $k = 1$ to K **do**
- 6: $\tau_k = 1 - (k - 1)/K, \quad \tau_{k+1} = 1 - k/K$
- 7: $z_{\tau_{k+1}} = z_{\tau_k} - (\tau_k - \tau_{k+1}) \cdot u_\theta(z_{\tau_k}, \tau_{k+1}, \tau_k, o)$
- 8: **end for**
- 9: $a = z_0$
- 10: **end if**

Algorithm 3 DMPO Stage 2: PPO Fine-Tuning

Require: Pre-trained policy π_θ , environment, hyperparameters $\lambda_V, \lambda_{\text{ent}}, \lambda_{\text{BC}}$
Ensure: Fine-tuned policy π_θ

- 1: **for** iteration $n = 1, \dots, N_{\text{iter}}$ **do**
- 2: Collect trajectories $\{(o_t, a_t^{0:K}, r_t)\}$ using current policy with K -step sampling
- 3: Compute GAE advantages $\{\hat{A}_t\}$ and returns $\{R_t\}$
- 4: **for** each PPO epoch **do**
- 5: Compute probability ratio: $\rho_t(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{\text{old}}(a_t|o_t)}$
- 6: Policy loss: $\mathcal{L}_{\text{PG}} = \mathbb{E}_t[\max(-\hat{A}_t \rho_t, -\hat{A}_t \text{clip}(\rho_t, 1 \pm \epsilon))]$
- 7: Value loss: $\mathcal{L}_V = \frac{1}{2} \mathbb{E}_t[(V_\theta(o_t) - R_t)^2]$
- 8: Entropy loss: $\mathcal{L}_{\text{ent}} = -\mathbb{E}_t[H(\pi_\theta(\cdot | o_t))]$
- 9: BC loss: $\mathcal{L}_{\text{BC}} = \mathbb{E}_t[\|a_\omega(o_t) - a_\theta(o_t)\|_2^2]$
- 10: Update BC coefficient: $\lambda_{\text{BC}}(n)$ via linear decay schedule
- 11: Total loss: $\mathcal{L} = \mathcal{L}_{\text{PG}} + \lambda_V \mathcal{L}_V + \lambda_{\text{ent}} \mathcal{L}_{\text{ent}} + \lambda_{\text{BC}}(n) \mathcal{L}_{\text{BC}}$
- 12: Update: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
- 13: **end for**
- 14: **end for**

D. 理论分析与统一框架设计

D.1. 分散正则化的理论分析

本小节为分散正则化提供了严格的信息论基础，解释了其为何能有效防止表示坍缩并提升单步生成质量。

预备知识与符号说明。 设 $\mathcal{O}$ 表示观测空间， $\mathcal{Z} \subseteq \mathbb{R}^d$ 表示表示空间，其中 d 为表示维度。编码器 $f_\theta : \mathcal{O} \rightarrow \mathcal{Z}$ 将观测映射为 d 维表示向量。对于一批 B 个观测 $\{o_i\}_{i=1}^B$ ，我们得到表示集合 $\mathbf{H} = \{\mathbf{h}_i = f_\theta(o_i)\}_{i=1}^B$ 。我们考虑两个随机变量：

- O ：服从数据分布 $p(o)$ 的观测随机变量
- $Z = f_\theta(O)$ ：由编码器诱导分布的表示随机变量

本文采用标准的信息论符号： $I(X; Y)$ 表示随机变量之间的互信息 X 与 Y ， $H(X)$ 表示香农熵 X ，且 $H(X | Y)$ 表示给定 Y 的条件熵 X 。

信息瓶颈视角。 从信息瓶颈（Information Bottleneck）视角来看，一个好的表示 Z 应当 satisfy:

$$\max_{f_\theta} I(Z; A | O) \quad \text{s.t.} \quad I(Z; O) \text{ 被最大化} \quad (40)$$

其中 A 为目标动作。第一项要求表示包含预测动作所需的信息，第二项要求表示充分编码观测信息。换言之，良好的表示 Z 应当是对 O 的充分压缩，同时保留预测 A 所需的全部信息。

关键观察： 平均流（MeanFlow）的速度预测损失 $\mathcal{L}_{\text{MF}}$ 仅优化第一项（预测准确率），而未对第二项（表示信息量）施加显式约束。这为表示坍缩留下了空间，模型可能学习到一种“捷径”表示 Z ，其预测损失较低，但丢弃了关键的观测信息。

互信息的熵分解。 互信息可分解为：

$$I(Z; O) = H(Z) - H(Z | O) \quad (41)$$

其中 $H(Z) = -\mathbb{E}_{z \sim p(z)}[\log p(z)]$ 是衡量表示分布离散程度的边际熵， $H(Z | O) = -\mathbb{E}_{o \sim p(o)}[\mathbb{E}_{z \sim p(z|o)}[\log p(z|o)]]$ 是给定观测下表示不确定性的条件熵。

对于确定性编码器 f_θ ，给定 o ，我们有 $z = f_\theta(o)$ 是确定性的，因此 $H(Z | O) = 0$ 。这意味着：

$$I(Z; O) = H(Z) \quad (42)$$

因此，最大化互信息 $I(Z; O)$ 等价于最大化边际熵 $H(Z)$ ，即让表示分布尽可能分散。

为什么平均流训练允许表示坍缩。 平均流训练目标（式 18）可以写为：

$$\mathcal{L}_{\text{MF}}(\theta) = \mathbb{E}_{(o,a), \epsilon, (r, \tau)} [\|u_\theta(z_\tau, r, \tau, o) - u_{\text{tgt}}\|^2] \quad (43)$$

这是一个纯回归损失，仅约束速度预测的准确率。考虑两种极端情况：

案例 1：分散表示。 不同观测 o_i, o_j 映射到不同表示 $\mathbf{h}_i \neq \mathbf{h}_j$ 。速度网络可以利用表示差异来区分样本，并生成精确的条件速度预测。

案例 2：坍缩表示。 所有观测映射到相似表示 $\mathbf{h}_i \approx \mathbf{h}_j$ ，但速度网络仍可能通过以下方式实现低训练损失：（1）过拟合训练数据中的统计模式，（2）利用时间嵌入 τ 和噪声状态 z_τ 中的信息，以及（3）在后续层中补偿表示信息损失。

神经网络优化表现出隐式正则化：梯度下降倾向于收敛到参数空间中的“简单”解。坍塌表示对应于有效复杂度较低的解，因为低秩协方差矩阵意味着表示仅沿少数方向变化，而大多数维度保持恒定或高度相关。在没有显式约束的情况下，优化过程可能收敛到此类退化解，因为它们需要较少的“活跃”参数来解释数据。

形式化表征： 设 $\mu = \mathbb{E}[\mathbf{h}]$ 为平均表示向量， $\Sigma_{\mathbf{H}} = \mathbb{E}[(\mathbf{h} - \mu)(\mathbf{h} - \mu)^\top]$ 为表示协方差矩阵。表示坍塌对应于秩 $(\Sigma_{\mathbf{H}}) \ll d$ ，意味着大多数特征维度高度相关或近似恒定。

分散正则化如何最大化熵

InfoNCE-L2 损失的熵解释： InfoNCE-L2 损失（公式 22）为：

$$\mathcal{L}_{\text{NCE-L2}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\|\mathbf{h}_i\|_2^2/\tau)}{\sum_{k \neq i} \exp(-\|\mathbf{h}_i - \mathbf{h}_k\|_2^2/\tau)} \quad (44)$$

其中 $\tau > 0$ 是温度参数，控制 Softmax 函数分布的锐度。

最小化该损失等价于：

1. 最大化分子：鼓励 $\|\mathbf{h}_i\|_2^2$ 增大，将表示推离原点
2. 最大化分母中的指数项：鼓励 $\|\mathbf{h}_i - \mathbf{h}_k\|_2^2$ 增大，将不同样本的表示推开

这两种效应共同促使表示在特征空间中分散，对应高熵分布。

与均匀分布的联系： 对于有限样本，最大化离散熵 $H(Z)$ 的分布是均匀分布。对于连续表示空间，分散性损失鼓励表示近似均匀地覆盖特征空间的某个区域，而不是聚集在少数点周围。设表示分布为 $p(z)$ ，均匀分布为 $q(z) = \text{Uniform}(Z)$ 。分散性损失可以理解为最小化 $p(z)$ 与某个高熵参考分布之间的距离。

协方差正则化的熵解释： 协方差损失（公式 25）为：

$$\mathcal{L}_{\text{Cov}} = \frac{1}{d_h} \sum_{i=1}^{d_h} \sum_{j \neq i} [C(\mathbf{H})]_{ij}^2 \quad (45)$$

其中 d_h 是表示向量的隐藏维度， $C(\mathbf{H}) \in \mathbb{R}^{d_h \times d_h}$ 是从批次 $\mathbf{H}$ 计算得到的样本协方差矩阵。记号 $[C(\mathbf{H})]_{ij}$ 表示该矩阵的第 (i, j) 个元素。

该损失惩罚协方差矩阵的非对角元素，促使 $C(\mathbf{H})$ 趋近于对角矩阵。对于一个 d 维高斯分布 $\mathcal{N}(\mu, \Sigma)$ ，其均值为 μ ，协方差为 Σ ，微分熵为：

$$H(\mathcal{N}(\mu, \Sigma)) = \frac{1}{2} \log \det(2\pi e \Sigma) = \frac{d}{2} \log(2\pi e) + \frac{1}{2} \sum_{i=1}^d \log \sigma_i^2 \quad (46)$$

其中 σ_i^2 表示 Σ 的第 i 个特征值（或当 Σ 为对角矩阵时的第 i 个对角元素）。第二个等式由行列式性质 $\det(k\mathbf{A}) = k^d \det(\mathbf{A})$ 得出，其中 k 为标量， $d \times d$ 为矩阵 $\mathbf{A}$ ：

$$\det(2\pi e \Sigma) = (2\pi e)^d \det(\Sigma) = (2\pi e)^d \prod_{i=1}^d \sigma_i^2 \quad (47)$$

取对数： $\log \det(2\pi e \Sigma) = d \log(2\pi e) + \sum_{i=1}^d \log \sigma_i^2$ ，乘以 $\frac{1}{2}$ 后即得该公式。从几何角度看， $\det(\Sigma)$ 表示特征空间中概率集中区域的体积：行列式越大，体积越大，意味着分布越分散，因此熵越高。在固定

总方差 $\sum_i \sigma_i^2$ 的条件下，当协方差矩阵为对角矩阵时熵最大，这由算术-几何平均不等式得出：乘积 $\prod_i \sigma_i^2$ （因此 $\det(\Sigma)$ ）在所有 σ_i^2 相等时达到最大，而不是某些大某些小。去相关使各维度独立，避免信息冗余，从而最大化有效信息容量。

联合训练目标：信息论视角。从信息论的角度来看，DMPO 第一阶段的总训练目标（式 26）可以理解为：

$$\mathcal{L}_{\text{Stage1}} = \underbrace{\mathcal{L}_{\text{MF}}}_{\text{minimize prediction error}} + \alpha_{\text{disp}} \underbrace{\mathcal{L}_{\text{disp}}}_{\text{maximize } H(Z)} \quad (48)$$

根据数据处理不等式（Data Processing Inequality），对于马尔可夫链 $O \rightarrow Z \rightarrow \hat{A}$ ，其中 $\hat{A}$ 表示预测动作：

$$I(O; \hat{A}) \leq I(O; Z) = H(Z) \quad (49)$$

该不等式的直觉在于：信息经过处理后只会减少（绝不会增加）——当数据经过任何变换时，部分信息可能丢失，但新信息无法凭空产生。因此，最大化 $H(Z)$ 为下游预测任务的信息量提供了上界保证。

为何一步生成放大了对色散正则化的需求。误差校正机制

多步推理：考虑 K 步推理，其中 K 为总去噪步数。设 δ_k 表示第 k 步的速度预测误差， $\Delta \tau_k$ 表示第 k 步的时间步长。最终动作中的累积误差为：

$$\epsilon_{\text{total}} = \sum_{k=0}^{K-1} \Delta \tau_k \cdot \delta_k \quad (50)$$

对于多步推理（大 K ），每步步长 $\Delta \tau_k = 1/K$ 较小，后续步骤可部分纠正先前误差。这为退化表示质量提供了鲁棒性。

一步推理的脆弱性：对于单步推理（ $K = 1$ ），整个状态转移在一步内完成：

$$a = z_1 - u_{\theta}(z_1, r = 0, \tau = 1, o) \quad (51)$$

在这种情况下：（1）步长 $\Delta \tau = 1$ 完全放大误差，（2）没有迭代校正的机会，（3）速度预测必须在单次前向传播中准确。

因此，单步推理要求表示 $\mathbf{h} = f_{\theta}(o)$ 为速度网络提供足够精确的条件信息，以便在单次前向传播中生成准确的动作。当表示坍塌时，不同观测的表示变得不可区分，速度网络无法生成正确的条件速度预测。

形式化分析：设 $d_{\text{eff}} = \text{rank}(\mathbf{\Sigma H})$ 表示表示的有效维度，即表示空间中非退化维度的数量。对于需要区分 N 种不同观测模式（例如，不同的操作状态或物体配置）的任务，理论上需要 $d_{\text{eff}} \geq \log_2 N$ 来唯一编码每种模式。这里 N 指的是策略必须区分的语义上不同的观测类别数量，而不是训练样本或表示向量的数量。当 d_{eff} 足够时，表示空间具有足够的容量来编码观测差异。然而，当坍塌导致 $d_{\text{eff}} \ll \log_2 N$ 时，表示无法区分不同的观测，速度预测退化为对所有输入的平均响应。

小结。MeanFlow 的训练目标仅包含速度预测误差，缺乏对表示结构的约束，使得坍塌的表示成为有效的局部最优解。该问题在少步推理中被放大，因为有限的迭代步骤无法弥补表示信息的损失。DMPO 通过分散正则化解决这一问题，它提供了：（1）信息论保证：最大化表示熵 $H(Z)$ ，从而最大化互信息 $I(Z; O)$ ，确保表示充分编码观测信息（式 42）。

（2）几何效应：在特征空间中引入排斥势能可防止表征聚类并保持分布均匀性。

D.2. 统一框架设计

一个潜在的担忧是，DMPO 的三个组成部分（平均流单步生成、分散正则化和近端策略优化微调）是否构成了缺乏原则性整合的“技术堆叠”。本小节将证明这些组成部分形成了一个连贯、相互依存的系统，源于对模型架构和算法设计的联合考量。

设计理念。DMPO 的核心目标是通过两个架构考量实现实时机器人控制：

（1）采用轻量级模型架构，以及（2）将采样步数减少到一步。然而，仅靠这些架构简化会导致复杂任务上的性能严重下降。因此，我们联合设计了算法改进，包括分散正则化和近端策略优化微调，以补充架构选择。

附录 C 和 D.1 节中的理论分析证明了为什么分散正则化对于稳定的一步生成是必要的，实验结果验证了其有效性。

功能依赖关系。三个组件之间存在严格的函数依赖关系，移除其中任何一个都会损害整个系统。若缺乏色散正则化，平均流（MeanFlow）的单步生成会遭遇表征坍缩，导致在多阶段任务上性能不稳定。若缺乏单步生成，多步推理会增加延迟，无法实现实时控制；此外，这会大幅降低近端策略优化（PPO）的训练吞吐量。若缺乏近端策略优化微调，策略将受限专家示范的质量，从而限制在次优示范任务上可达到的性能。这种相互依赖关系使 DMPO 区别于简单的技术组合。

验证。理论分析与实验结果均证实了所有组件的必要性。消融研究表明，移除任一组件都会显著降低任务成功率，或无法达到实时控制的推理时间。表征多样性与任务成功率之间的强相关性验证了色散正则化直接促成稳定的单步生成。DMPO 相较于多步基线实现了显著加速，同时达到或超越其最终性能，表明联合架构-算法设计成功解决了效率与性能之间的权衡问题。